

Secure Multiparty Computation Meets Deep Learning

Yoshi234

2024-01-30

Table of contents

Preface	6
Contents	6
Resources	6
I Secure Multiparty Computation Primitives	7
1 Beaver's Triples Explained	9
1.1 Generating Boolean Beaver's Triples	9
1.1.1 Step 1: Generating Inputs	9
1.1.2 Step 2: Obtaining Shares of c	10
1.2 Application of Random Beaver's Triples	10
2 Oblivious Transfer	12
2.1 What is Oblivious Transfer?	12
2.2 Extending Oblivious Transfer	13
2.3 Fundamentals of Oblivious Transfer	13
2.3.1 Random Oblivious Transfer (ROT)	13
2.3.2 Correlated Oblivious Transfer (COT)	14
2.3.3 IKNP	14
3 One-Way-Functions	15
4 CrypTen	16
4.1 Encrypting Models with CrypTen	16
II Deep Learning	17
5 Word Embeddings and Word2Vec	19
6 Recurrent Neural Networks	21
6.0.1 Reasons NOT to use RNNs	21
7 Long Short-Term Memory Networks	25
7.0.1 Main Ideas	25

8	Sequence-to-Sequence (seq2seq) Encoder-Decoder Neural Networks	27
8.0.1	Unique features of Encoder-Decoder Training	28
9	Attention for Neural Networks	29
10	Residual connections	31
10.1	Motivation for Residual connections	31
10.2	Formulation	31
10.3	The Utility of Residual Blocks for Training Deep Networks	31
10.3.1	Ensemble of Shallow Neural Networks	32
11	Object Detection Algorithms	33
11.1	SSD: Single Shot MultiBox Detector	33
11.2	A Comprehensive Review of YOLO (v1 to v8+)	33
11.2.1	Applications of YOLO	34
11.2.2	Evaluation Metrics	34
11.2.3	Non-Maximum Suppression	35
11.2.4	YOLO	36
11.2.5	PP-YOLO Models	53
11.2.6	Summary of YOLO	55
12	Implementation Guides for Yolo Object Detectors	56
12.1	YOLOv5 Architecture In Depth	56
13	AdderNet	57
13.1	Defining Adder-Networks	57
13.1.1	Convolution v. L1-Distance	58
III	V2X Applications	59
14	Red Light Violation Detection	61
14.1	Motivation	61
14.2	V2I Algorithms for RLR Detection	61
14.2.1	Red Light Violation Detection Algorithm	61
14.3	Thao et.al on Traffic Violation Detection	62
14.3.1	Problem Setting	62
14.3.2	System Design and Solution Approach	62
14.3.3	Primary Contributions	64
14.4	Goma et.al on RLR Detection with SSD (Single Shot Detector)	64
14.4.1	Methods and System Design	65

15 Traffic Flow Forecasting	66
15.1 Spatio-Temporal Graph Convolutional Networks (STGCN)	66
15.1.1 Graph Convolutions	66
15.1.2 Network Architecture	67
15.1.3 Graph Convolution Approximations	68
15.1.4 Gated CNNs - temporal feature extraction	69
15.1.5 Spatio-Temporal Convolutional Block	69
15.1.6 Prediction Task	70
15.1.7 Experimental Evaluation / Study	70
15.1.8 Topic Study Shortlist	71
15.1.9 What are Gated Convolutions?	71
15.1.10 Approximated Convolutions (Chebyshev Polynomial Approximations)	72
15.1.11 What is linear interpolation?	72
15.2 Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting	72
15.2.1 Core Contributions of ASTGCN	73
15.2.2 Primitives	74
15.2.3 Problem Setting	74
15.2.4 The ASTGCN Model Framework	74
15.2.5 Things I don't Understand ...	76
15.3 Graph WaveNet	76
15.3.1 Contributions	76
15.3.2 Methods	76
15.3.3 Experiments and Evaluation	77
15.3.4 Questions	77
15.3.5 Ideas	79
15.4 ST-SSL	79
15.5 STPGCN	79
15.5.1 Contributions	79
15.5.2 Methods	80
15.5.3 Evaluation	84
15.5.4 Questions	84
16 Trajectory-Based Traffic Forecasting	85
16.1 T-Wave	85
16.1.1 Contributions	85
16.1.2 Methods	85
16.1.3 Experiments	85
16.1.4 Questions	85
16.2 MA2GCN	85
16.2.1 Contributions	85
16.2.2 Methods	86
16.2.3 Experiments / Evaluation	86

16.2.4 Questions	86
17 Summary	87
References	88

Preface

This is a Quarto book. To learn more about Quarto books visit <https://quarto.org/docs/books>.

Contents

The quarto book contains an organized structure of notes on a variety of secure multiparty computation protocols, implementation details, and a discussion of v2x applications and relevant deep learning models. The authors hope that any readers find these resources useful.

Resources

The [matcha editor](#) is used to construct some of the mathematical diagrams shown in this book. In order to export a matcha diagram, you need to enter the full-screen mode for the diagram, and click on the “export” drop-down which becomes available. This will allow you to save the math diagram as a png image. We will make liberal use of these throughout the book, especially for explaining complex security primitives such as beaver’s triples and homomorphic encryption.

Part I

Secure Multiparty Computation Primitives

This section of the text discusses a number of secure multiparty computation protocols in-depth and provides sample implementation to get you started.

1 Beaver's Triples Explained

Johua Lee
Ali Arastehfard

In order to mask fixed-point multiplication inputs, we need to use a secure primitive known as Beaver's triples. These were originally introduced by Beaver (1991) in his paper: [Efficient Multiparty Protocols Using Circuit Randomization](#)

1.1 Generating Boolean Beaver's Triples

The first step in masking the AND operation is to generate Boolean Beaver's Triples. These are random bits a , b , and c such that $c = a \cdot b$, where $\cdot$ represents the AND operation. Throughout this tutorial we define $a = a_A \oplus a_B$, $b = b_A \oplus b_B$, $c = c_A \oplus c_B$, and $r = r_A \oplus r_B$.

1.1.1 Step 1: Generating Inputs

Suppose we have two parties, **Alice** and **Bob**. We begin by first generating a random pair (a_i, b_i) for each party where i corresponds to the party in question. We generate these from the field F_2^2 which simply means that we select two random bits from $\{0, 1\}$.

$$\begin{array}{cc} \text{Alice} & \text{Bob} \\ \mathbb{F}_2^2 \xrightarrow{\$} (a_A, b_A) & \mathbb{F}_2^2 \xrightarrow{\$} (a_B, b_B) \end{array}$$

Next, Bob and Alice each generate random masking bits r_A and r_B respectively.

$$\begin{array}{cc} \text{Alice} & \text{Bob} \\ \mathbb{F}_2 \xrightarrow{\$} r_A & \mathbb{F}_2 \xrightarrow{\$} r_B \end{array}$$

1.1.2 Step 2: Obtaining Shares of c

After generating the masking bits, Alice and Bob invoke an instance of $\binom{2}{1}$ -OT₂ (see Chapter 2), where Alice acts as the sender with inputs $(r_A, r_A \oplus a_A)$, and Bob acts as the receiver with input b_B . Bob obtains $b_B a_A \oplus r_A$.

Now, Alice and Bob invoke another instance of $\binom{2}{1}$ -OT₂. This time, Bob serves as the sender with inputs $(r_B, r_B \oplus a_B)$, and Alice serves as the receiver with input b_A . Consequently, Alice obtains $b_A a_B \oplus r_B$.

$$\begin{array}{ccc}
 \text{Alice} & & \text{Bob} \\
 \text{Snd} = (r_A, r_A \oplus a_A) & \xrightarrow{\binom{2}{1}\text{-OT}_2} & \text{Rcv} = b_B a_A \oplus r_A \\
 \text{Rcv} = b_A a_B \oplus r_B & \xleftarrow{\binom{2}{1}\text{-OT}_2} & \text{Snd} = (r_B, r_B \oplus a_B)
 \end{array}$$

Note that objective is for Alice and Bob to each hold shares of c , denoted as c_A and c_B , respectively, such that their combination satisfies the equation $c = ab$.

Alice had received $b_A a_B \oplus r_B$ from Bob, and she already has $b_A a_A \oplus r_A$. She computes $b_A a_B \oplus r_B \oplus b_A a_A \oplus r_A$, which simplifies to $b_A a \oplus r$. Alice sets $c_A = b_A a \oplus r$.

Similarly, Bob computes $b_B a_A \oplus r_A \oplus b_B a_B \oplus r_B$, which simplifies to $b_B a \oplus r$. Bob sets $c_B = b_B a \oplus r$.

$$\begin{array}{ccc}
 \text{Alice} & & \text{Bob} \\
 c_A = b_A a \oplus r & & c_B = b_B a \oplus r
 \end{array}$$

It is easy to see that $c = c_A \oplus c_B = b_A a \oplus r \oplus b_B a \oplus r = ab$.

Thus, we have shown how Boolean Beaver's triples are randomly generated such that $c = c_A \oplus c_B$.

1.2 Application of Random Beaver's Triples

Now that randomized triples (a_A, b_A, c_A) and (a_B, b_B, c_B) have been generated for Alice and Bob respectively, we can use them cooperatively compute the output of some multiplication $x \cdot y$.

Suppose that Alice has shares of x and y (x_A, y_A) and Bob has shares (x_B, y_B) .

Alice	Bob
$f_A = y_A \oplus b_A$	$f_B = y_B \oplus b_B$
$e_A = x_A \oplus a_A$	$e_B = x_B \oplus a_B$
$f = y \oplus b$ $e = x \oplus a$	

Now that functional shares have been computed, we can use those to compute shares z_A and z_B for collectively obtaining the value of $x \cdot y$

Alice	Bob
$z_A = f \cdot a_A \oplus e \cdot b_A \oplus c_A$	$z_B = f \cdot a_B \oplus e \cdot b_B \oplus c_B$

Note: in the computation above, we approach the binary case of multiplication.
Thus, the $\cdot$ operation is equivalent to the $\cap$ operation

We **XOR** these shares z_A and z_B together to get the following

$$\begin{aligned}
z_A \oplus z_B &= [(y \oplus b) \cdot (x \oplus a) \oplus (y \oplus b) \cdot a] \oplus [(x \oplus a) \cdot b \oplus c] \\
&= (y \oplus b)(x \oplus a) \oplus xb \oplus ab \oplus c \\
&= (y \oplus b)x \oplus xb \\
&= yx \oplus xb \oplus xb = yx
\end{aligned}$$

2 Oblivious Transfer

2.1 What is Oblivious Transfer?

If you're not familiar with Boolean operation XOR, please reference the following truth table:

A	B	$A \oplus B$
0	0	0
1	0	1
0	1	1
1	1	0

In standard computing practice, we often run into the issue of wanting to transfer values between parties. This problem can be modeled as follows:

Suppose that Alice has a set of values $\{s_1, s_2, \dots, s_n\}$ and Bob holds a selection bit i . Bob would like to obtain the value of s_i from Alice.

Without accounting for security, this can easily be accomplished by Bob sending his selection bit to Alice, and Alice sending the corresponding value s_i to Bob or Alice sending her entire set of values to Bob, and Bob picking the desired value s_i from the obtained set.

However, in Secure MPC (multi-party computation) we are concerned with the security of this process. More specifically, it is important that *Bob doesn't learn anything about Alice's s_i 's other than the one he receives* and *Alice doesn't learn anything about Bob's selection bit i* . Oblivious transfer is the secure mechanism by which we achieve this outcome. This is an extremely important Secure MPC tool because it has been proven that **OT is complete for all Secure MPC operations** (see O. Golrich (1987) and Killian (1988)).

As such, OT forms the basis for many secure computation protocols which have been developed in the past 40 years or so.

2.2 Extending Oblivious Transfer

Due to the the time complexity associated with Oblivious Transfer, researchers have been motivated to search for efficiency gains. Towards this end, several important developments have been made over time:

- It was demonstrated by R. Impagliazzo (1989) that if a reduction of OT to one-way-functions (black-box reduction) could be performed, it would imply that $P \neq NP$ (polynomial-class algorithm problems and NP class problems are not complexity-equivalent)
 - in other words, OT is harder than one-way-functions which form the basis of symmetric key encryption. See Chapter 3 for more information on this (OWF) one-way-functions.
- Beaver demonstrated one could extend a series of small base OT's to a large number of OT's using ONLY OWF's Beaver (1996)
- IKNP OT - (proposed in Y. Ishai (2003))
 - uses ROM (random oracle model) and symmetric key primitives to increase security and improve efficiency of OT
- Silent OT (Ferret) - (proposed in K. Yang (2020))
- Exponential Correlated OTs based on LPN Assumption (proposed in E. Boyle (2020))

2.3 Fundamentals of Oblivious Transfer

2.3.1 Random Oblivious Transfer (ROT)

what is the difference between ROT_l^m and OT_l^m ?

Random Oblivious Transfer is the most general form of OT. In this setting, we assume that there are two parties involved: Alice and Bob. Here, Alice functions as the *sender* and Bob functions as the *receiver*. Additionally, we assume a 1-out-of-2 ROT functionality. Alice and Bob follow the ROT_m^l protocol below:

The receiver (Bob) provides its selection bit b to the protocol, and two random strings are generated by the ROT protocol. These are r_0 and r_1 , one of which corresponds to the input bit b . The protocol provides these two random strings to the sender, and provides the receiver with a bit $r_b \in \{r_0, r_1\}$ which corresponds to the receiver's selection bit.*

How are bits selected by the protocol without assistance of a TTP?

Case 1: Bob uses the same selection bit b

Once the random outputs have been generated for Alice and Bob, they collectively perform an instance of $\binom{2}{1}$ -OT.

1. Alice inputs chosen bits (x_0, x_1) and Bob inputs selection bit b
2. Alice sends to Bob $(x_0 \oplus r_0, x_1 \oplus r_1)$
3. Bob performs $(x_b \oplus r_b) \oplus r_b$ to obtain x_b

Here, only one message is passed from Alice to Bob $S \rightarrow R$

Case 2: Bob chooses a different selection bit c

1. Bob sends to Alice $k = b \oplus c$ where b was the chosen bit input to the ROT protocol, and c is the chosen bit of the $\binom{2}{1}$ -OT
2. Alice permutes k with r_0 and r_1 to get r_k and $r_{\bar{k}}$.
3. Alice sends $(y_0, y_1) = (x_0 \oplus r_k, x_1 \oplus r_{\bar{k}})$ to Bob
4. Bob performs $y_c \oplus r_b$ to obtain x_c

2.3.2 Correlated Oblivious Transfer (COT)

Correlated Oblivious transfer has functionality defined as follows:

- The sender (Alice) has values x_0, x_1 such that the $\oplus$ of these values together results in δ
- Alice inputs Δ and receives r (a random l -bit string) and $r \oplus \Delta$.
- The receiver (Bob) obtains $r \oplus b\Delta$.

We use the notation COT_k^m to represent the correlated oblivious transfer functionality. Here m is the number of OT's to perform, and k is length of the random bit-strings used to mask the values exchanged.

In this setting, the ROT_l^m problem can be reduced to COT_k^m . If a random oracle is given, we can hash the output of a k length output to an l length output for ROT_l^m . However, it is not strictly necessary to use a random oracle. A correlation robust hash function may be used given that the joint distribution of hashes $(h(t_1 \oplus s), h(t_2 \oplus s), \dots, h(t_m \oplus s))$ for strings $s, t_1, \dots, t_m$ given $t_1, \dots, t_m$ is pseudo-random.

2.3.3 IKNP

3 One-Way-Functions

4 CrypTen

CrypTen is a secure multiparty computation framework introduced by B. Knott (2022). CrypTen is significant because it takes concepts which are well-established within the context of multiparty computation, and establishes an easy-to-use system which will feel familiar to researchers already know PyTorch. This is extremely important since the widespread adoption of secure computation techniques up to this point have largely been limited to (1) the complexity of primitives associated and (2) the low-level implementation used by most state-of-the-art systems such as Delphi and Cheetah.

4.1 Encrypting Models with CrypTen

In other systems such as Delphi, model encryption is treated as an issue of

Part II

Deep Learning

This portion of the text deals with deep learning and artificial intelligence applications. Specifically, it covers topics such as object detection and classification with computer vision, neural network primitives, and others. Other machine learning applications and import resources may also be included in this portion of the text

5 Word Embeddings and Word2Vec

These notes are based on Josh Starmer's video [Word Embedding and Word2Vec, Clearly Explained!!](#) on YouTube.com.

We need a method for converting words into numeric values. Otherwise, we won't be able to utilize them within a machine learning algorithm. We also want a method for representing words that understand the similarity between similar words - and mitigate the distance between those representations (they should be close together in a vector space if their semantic meanings are similar)

A very simple approach to understanding this problem is as follows:

Suppose you are given two phrases: "Troll 2 is great!" and "Gymkata is great!". In this case, there are four separate words ("Troll 2" is treated as a single word). Then, connect each of the inputs to at least one activation function. The number of activation functions for each input is equivalent to the number of components to associate with each word. *The weights on the connection to the each activation function represents the embedding values for a particular word.* Initially, we use random weights, but backpropagation can be used to predict the prior word.

Our prediction task here is the word which directly follows the input word. So, if the input is "Troll 2", then we want "is" to be the output word (the strongest predicted value). Then, connect the activation functions to outputs, and add weights to those connections. Then, the outputs are run through the softmax classification function. Crossentropy loss can then be used for backpropagation.

Additional context is used to improve the embeddings learned

1. *Continuous Bag of words* - the inputs are the words which occur before and after the target word, and the output is the word which occurs between them. For example, in "Troll 2 is great!", we want to embed "is" using the context "Troll 2" and "great!"
2. *Skip Gram* - the word in the middle is used to predict the surrounding word.

In practice, more than one activation is used in order to train embeddings for words, and the training dataset will be much larger. This results in much larger embeddings for most words.

Negative Sampling

Negative sampling is used in order to improve the speed at which embeddings can be trained. Suppose we have the input

$$\langle \text{"A"} = 1, \text{"aardvark"} = 0, \dots, \text{"zigzag"} = 0 \rangle$$

Then, we can ignore all of the weights which come from each of the other words and only include weights for the word of interest. However, there will still be weights which connect the activation layer to the output layer for each word

6 Recurrent Neural Networks

These notes are based on Josh Starmer's video

In order to work with time series data to predict the following point, we need to use a model which account for a varying amount of inputs. For example, in stock market price prediction, we want to inputs with a varying amount of input values available for each.

Just like other neural network architectures, RNNs take an input and feed it through an activation. However, RNNs loop the input multiple times through the activation in order to obtain an output.

Alternatively, we can think of this process as unrolling the network for sequential time series points.

Suppose:

$$\begin{aligned}t_1 &\rightarrow ReLU(W_{11}t_1 + b_{11}) = y_1 \\t_2 &\rightarrow ReLU(W_{12}t_2 + b_{12} + y_1) = y_2 \\&\vdots \\W_{22}y_2 + b_2 &= \text{predicted value}\end{aligned}$$

Notice that the output for t_1 is essentially used as a bias for the second input. This process can be extended to an arbitrary number of sequence numbers. **NOTE:** even though we unroll the network, the weights and biases are shared!! This means that we never increase the number of weights and biases to train.

6.0.1 Reasons NOT to use RNNs

The more we unroll a network, the harder it becomes to train the network. This leads to the *vanishing and exploding gradient problem*.

For example, let's suppose some weight is given for W_1 (in all unrollings of the network) as 2. Then, every time we run the input, we perform $2 \times y_{i-1}$ which eventually gets us $2^n \times inputs$ which ends up almost always being a gigantic number, especially for values of n which any reasonable size, say $n > 50$ or so. This huge number makes it extremely difficult to take small

Recurrent Neural Network

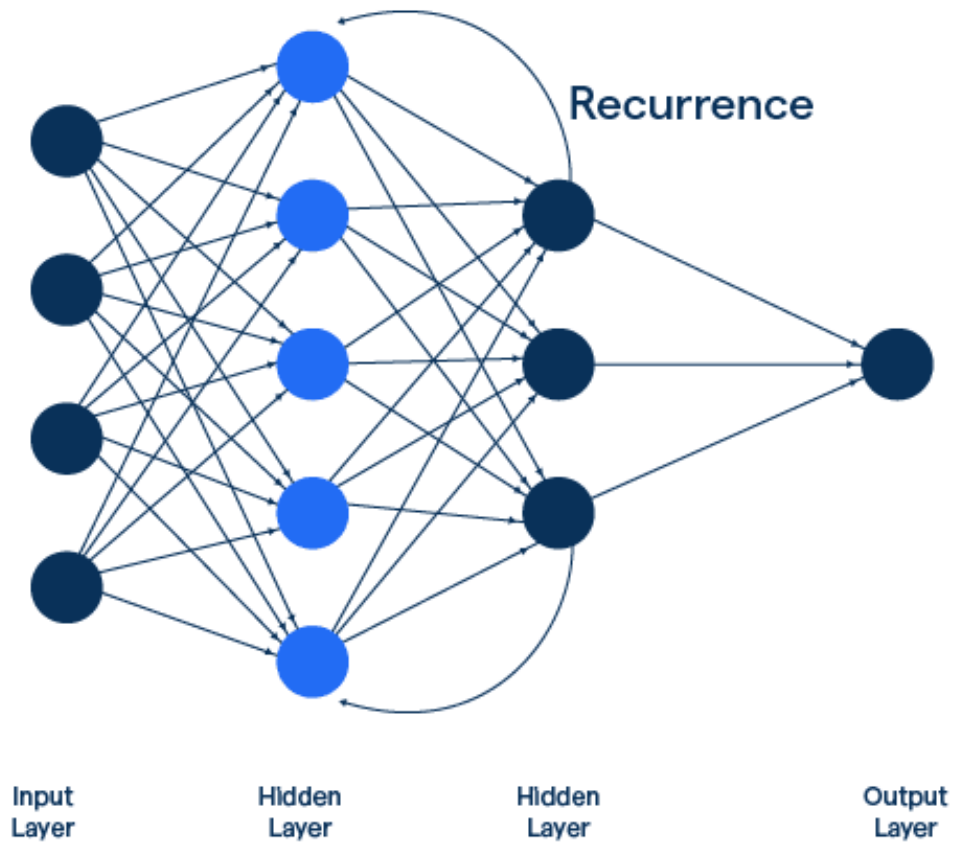


Figure 6.1: A simple RNN

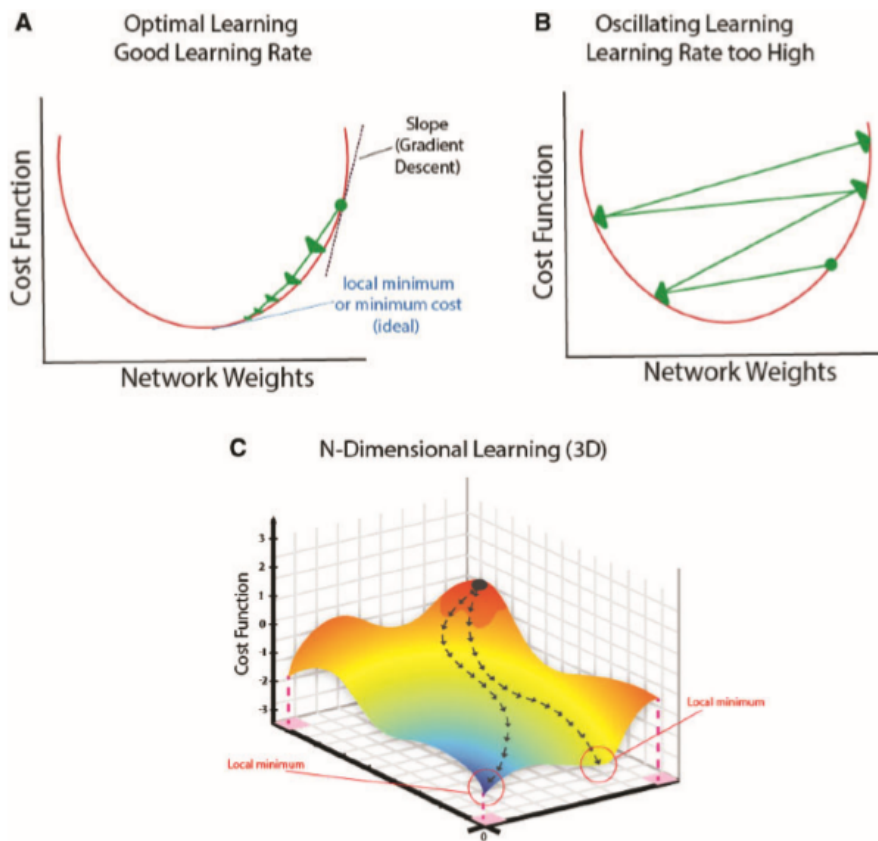


Figure 6.2: vanishing and exploding gradient problem

steps towards the correct parameter value. This means that instead of taking small steps, we will end up with extremely large jumps, meaning that we will bounce. However, if you use weight values which are less than 1, then you will obtain values which are extremely small, meaning that the maximum number of training steps will be reached before finding the optimal values.

LSTMs can be used to largely avoid this problem. See [Chapter 7](#)

7 Long Short-Term Memory Networks

These notes are based on Josh Starmer's video [Long Short-Term Memory \(LSTM\), Clearly Explained](#) on YouTube.com.

LSTMs are a type of recurrent neural network which is specifically designed to avoid the exploding and vanishing gradient problem (see Section 6.0.1).

7.0.1 Main Ideas

Suppose you have a time series problem, where multiple time stamps are given for each point in sequence. Then for each time stamp in the point, we re-run the output of the $i - 1$ timestamp through the activation $ReLU(Wx^T + b)$ and add it as a bias to the output of the i th timestamp. In the case of LSTMs, a more complex base unit is used in order to achieve the goals of the methodology.

Main Differences from RNN

- Basic units incorporate *sigmoid* and *tanh* activation functions
- cell-state - is an unweighted
- hidden-state (short-term memory) has modifiable weights.
 - the output from the hidden-state determines how much of the long-term memory is remembered. This is usually called the *forget gate*
 - starting with the rightmost block, we determine what memory should be remembered. (*tanh* activation)
 - then, the left block determines the percentage of the short-term memory to remember.
 - This new long term memory update is called the *input gate*

Updating short-term memory

The output from the long-term memory is used as an input to the short-term memory state. Then, a data (time-series) input is provided, and the output is determined from a *tanh* activation function. This final step is known as the *output gate*

When you begin running an LSTM, you set the initial long and short term memory to 0. The final output of the short term memories is the prediction of the associated value.

NOTE: the LSTM uses the same wieghts for every unrolling of the network. This property is preserved from standard RNNs.

8 Sequence-to-Sequence (seq2seq) Encoder-Decoder Neural Networks

These notes are based on Josh Starmer's video [Sequence-to-Sequence \(seq2seq\) Encoder-Decoder Neural Networks, Clearly Explained!!!](#)

Sequence-to-sequence problems - when you want to translate one sequence into a sequence of another kind, then you have a sequence-to-sequence problem. seq2seq models can be used to address these sorts of problems.

To understand these models better, take the following example. Suppose you want to translate the sentence, "Let's go." into Spanish - "Vamos."

- *challenge 1*: different sentences can be of different length, so the model needs to be capable of accepting inputs of multiple lengths.
- *challenge 2*: the output must be able to accomodate a varying length sentence.

LSTMs can be used to address this problem. Namely, you would need to plug "Let's" and then "go" into the LSTM sequentially. Also, since the vocabulary of words we train our model on incorporate a variety of words and symbols, we call these elements "tokens". Of course, words are used through vector embeddings of words to avoid as much computational overhead as less efficient methods.

Of course, additional LSTM cells can be added (horizontally) in order to increase the number of weights and biases. Additional layers (vertical) of LSTMs can also be added.

- *layers* - additional LSTM cells which are added on top of the previous layer. They are connected sequentially, and take inputs from the short-term memory outputs of the LSTM cell which sits beneath them. This short-term memory output moves forward to the next cell in the layer too.

These units form the encoder portion of the "encoder-decoder" model. The last long-term memories from the last cells of all layers of the encoder portion form the context vector which initializes the long and short-term memories of the decoder LSTMs.

The decoder portion of the network is comprised of an identical number of layers and cells (it's not clear to me at this time whether this is a required behavior of seq2seq networks) but has separate weights and biases from the encoder. The ultimate goal of the decoder is to decode the context vector from the encoder to the output sentence.

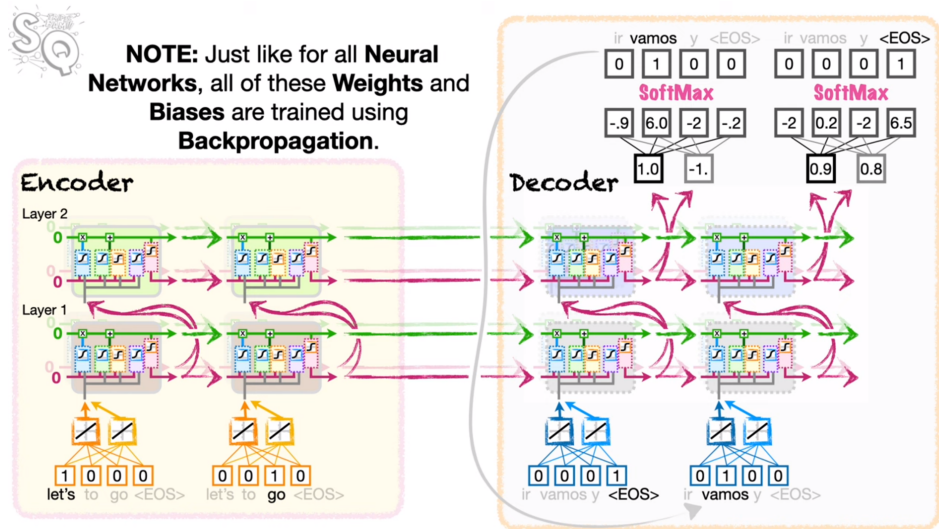


Figure 8.1: A simple encoder decoder network

Just as in the encoder, the input to the first activation of the LSTM cell is an embedding layer (create a vector embedding of the word). Once the LSTM is run, the output of the top cell of the final layer is run through a fully connected layer to outputs. The FC layer as an input for each of the outputs in the LSTM (long and short-term memory) and an output for each of the tokens in the Spanish vocabulary, and softmax classification is used to determine what the output word is. This word is actually fed back to the next cell in the first layer of the decoder which embeds the output as an input to the subsequent LSTM cells. The decoder will unroll the two LSTM cells (in the first and second layers of the decoder - reading inputs) until a token has been output by the decoder. Alternatively, the decoder will stop when the maximum output length is reached.

8.0.1 Unique features of Encoder-Decoder Training

During inference with encoder-decoders, the output of the LSTM cell in the top layer is used as input to the embedding layer for the next set of LSTM cells. However, during training, the actual next word embedding is used instead, rather than the predicted word embedding. Additionally, each output phrase ends where the known phrase ends! This method is known as *Teacher forcing*.

9 Attention for Neural Networks

These notes are based on Josh Starmer's video [Attention for Neural Networks, Clearly Explained!!!](#)

Issues with the basic encoder-decoder architecture

When we unroll the LSTMs in the encoder-decoder architecture, we end up compressing the entire input sentence into a single context vector. This is ok for short phrases, but for bigger input vocabularies this might not work. For example, words which come earlier on may be forgotten. For example, this would be a big issue if the first word at the beginning of “Don’t eat the pizza while petting the cat”.

The Novelty of Attention

Instead of having information forgotten by the LSTMs, new paths are created from older units (further towards the front) directly to the decoder so they can serve as inputs. However, adding these extra paths is not a simple process. This style of model is consistent for all encoder-decoder models with attention.

The Process of Attention

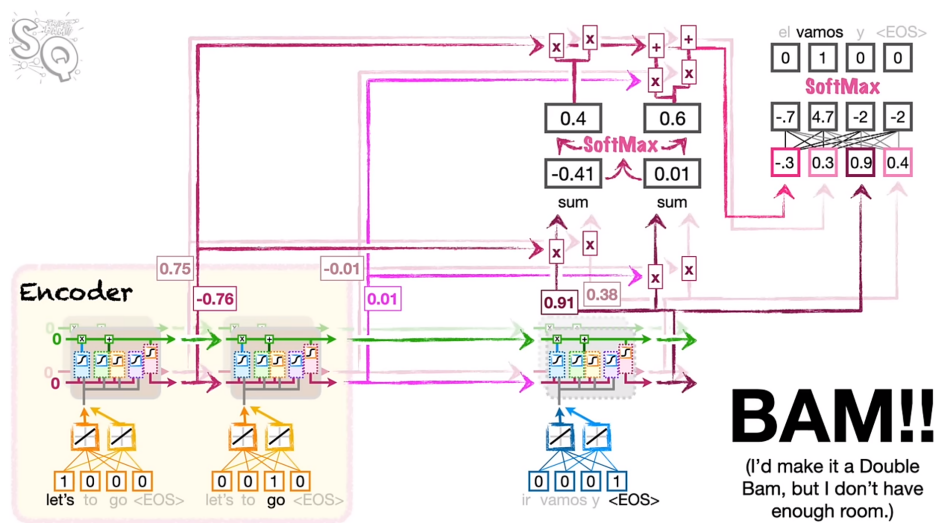


Figure 9.1: Attention step added to Seq2Seq

- attention determines how similar the outputs from the encoder LSTMs are at each step. (a similarity score between LSTM outputs and the first timestep of the decoder)
- There are many ways to determine the similarity of two word embeddings

- cosine-similarity:

$$\text{cosine similarity} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- * the numerator calculates the similarity between two sequences of numbers
 - * the denominator scales the value to between 0 and 1.
 - * alternatively, the numerator can be used alone - the scaling may or may not be necessary in all cases. This numerator is simply referred to as the dot product. this metric is more common because it is
- If the score for one word is higher, we want that word to have more influence on the first word that comes out of the decoder.
 - scores are run through the softmax function, and the output determines the percentage of the decoding accounted for by each of the input words. This output is run through a fully connected layer and softmax is used to determine the resulting word.

Just like with encoder-decoder networks, you keep using the next word, unrolling the network, if the predicted word is not <EOS>

However, now that attention has been added to the model, it actually turns out that the LSTM modules are not necessary.

10 Residual connections

This section discusses residual connections based on the information provided by Wong (2021) in his medium article

10.1 Motivation for Residual connections

Deep Neural Networks such as YOLO (see Section 11.2.4.3) allow for greater accuracy and performance. However, deep networks like this make it more difficult for the model to converge during training.

Residual connections help to make training networks easier.

10.2 Formulation

Residual connections - allow data to reach other parts of a sequential network by skipping layers

This flow is depicted well by the following image:

Steps:

1. Apply identity mapping to x - perform element-wise addition $F(x) + x$: this is the **residual block**
 1. residual blocks may also include a ReLU activation applied to $F(x) + x$. *This works when dimensions of $F(x)$ and x are the same
 2. If dimensions of $F(x)$ and x are NOT the same, then you can multiply x by some matrix of constants W to scale it. $F(x) + Wx$

10.3 The Utility of Residual Blocks for Training Deep Networks

Empirical results have demonstrated that residual blocks increase the speed and ease of network convergence. There are a number of suspected reasons as to why this enables such performance gains.

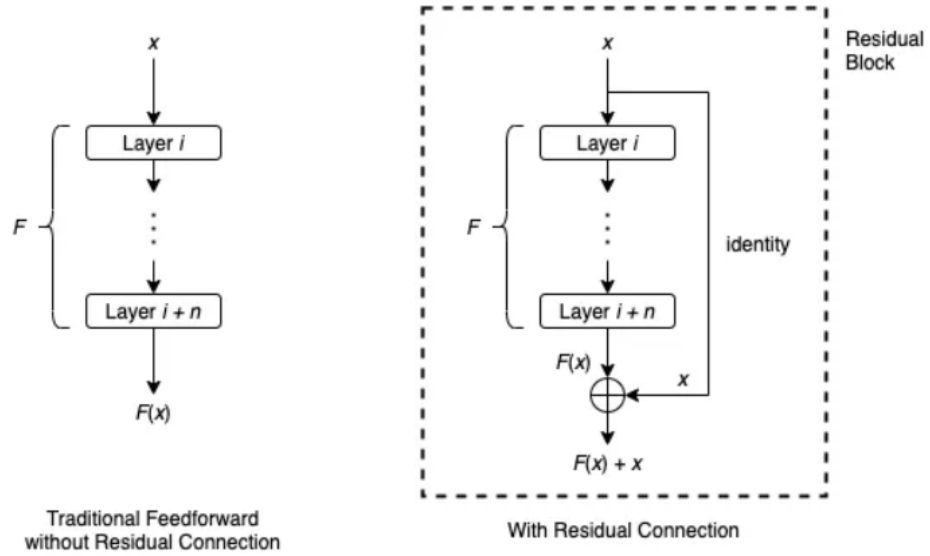


Figure 10.1: residual connection diagram

10.3.1 Ensemble of Shallow Neural Networks

10.3.2

11 Object Detection Algorithms

11.1 SSD: Single Shot MultiBox Detector

The single shot multibox detector is an algorithm presented by W. Liu (2016) for the purpose of taking a 300 x 300 input and generating bounding boxes on objects of interest within the image. The paper is linked [here](#)

11.2 A Comprehensive Review of YOLO (v1 to v8+)

J. Terven (2023) present review and analysis of the evolution of the Yolo algorithm, with a focus on the innovations and contributions made by each iteration, as well as the major changes in network architecture (and training tricks) which have been implemented over time.

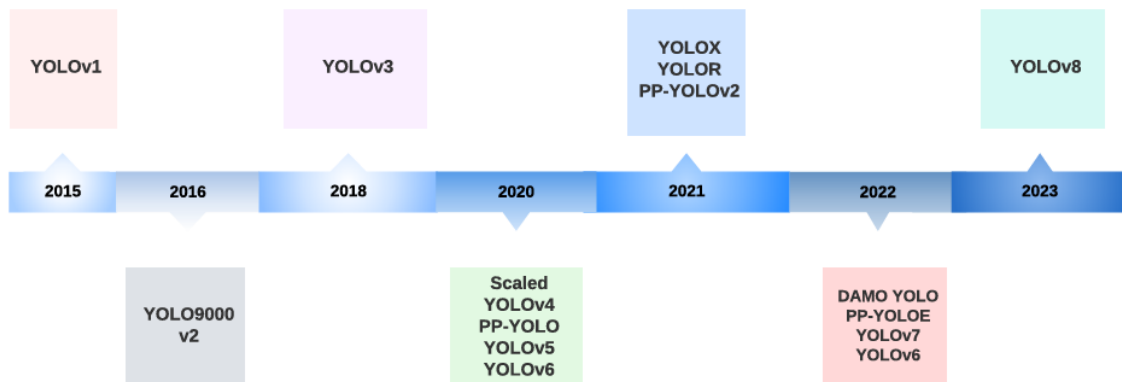


Figure 1: A timeline of YOLO versions.

Figure 11.1: A timeline of YOLO development

11.2.1 Applications of YOLO

Yolo has proven invaluable for a number of different applications

- autonomous vehicles
 - enables quick identification and tracking of objects like vehicles, pedestrians, bicycles and other obstacles
- action recognition
- video surveillance
- sports analysis
- human-computer interaction
- crop, disease, pest detection and classification
- face detection - biometrics, security, facial recognition
- cancer detection
- skin segmentation
- pill identification
- remote sensing
 - satellite and aerial imagery object detection / classification
 - land use mapping
 - urban planning
- security systems
- smart transportation systems
- robotics and drones

11.2.2 Evaluation Metrics

Average Precision (AP) and Mean Average Precision (mAP) are the most common metrics used in the object detection task. It measures average precision across all categories, providing a single value to compare different models

11.2.2.1 How AP works

- mAP is the average precision for accuracy of predictions across all classes of objects contained within an image
 - individual AP values are determined for each category separately.
- IOU (intersection over union)
 - measures the proportion of the predicted bounding box which overlaps which overlaps with the true bounding box

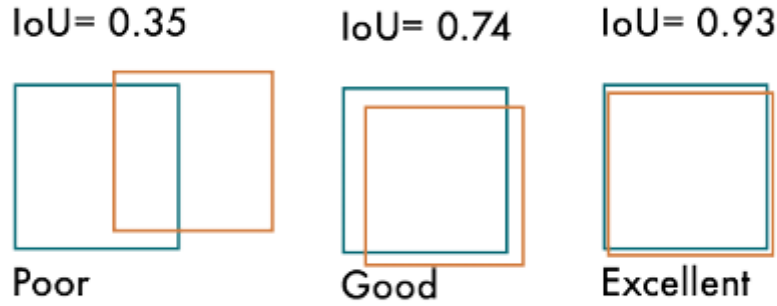


Figure 11.2: Intersection over union in practice

Different methods are used to compute AP when evaluating object detection methods on the COCO and VOC datasets (PASCAL-VOC)

11.2.3 Non-Maximum Suppression

A post-processing technique - reduces number of overlapping boxes and improves detection quality. Object detectors typically generate multiple bounding boxes around the same object. Non-max suppression picks the best ones and gets rid of the others.

The algorithm for this is defined below:

Algorithm 1 Non-Maximum Suppression Algorithm

Require: Set of predicted bounding boxes B , confidence scores S , IoU threshold τ , confidence threshold T

Ensure: Set of filtered bounding boxes F

```

1:  $F \leftarrow \emptyset$ 
2: Filter the boxes:  $B \leftarrow \{b \in B \mid S(b) \geq T\}$ 
3: Sort the boxes  $B$  by their confidence scores in descending order
4: while  $B \neq \emptyset$  do
5:   Select the box  $b$  with the highest confidence score
6:   Add  $b$  to the set of final boxes  $F$ :  $F \leftarrow F \cup \{b\}$ 
7:   Remove  $b$  from the set of boxes  $B$ :  $B \leftarrow B - \{b\}$ 
8:   for all remaining boxes  $r$  in  $B$  do
9:     Calculate the IoU between  $b$  and  $r$ :  $iou \leftarrow IoU(b, r)$ 
10:    if  $iou \geq \tau$  then
11:      Remove  $r$  from the set of boxes  $B$ :  $B \leftarrow B - \{r\}$ 
12:    end if
13:  end for
14: end while

```

Figure 11.3: Non-Max Suppression Alg

A useful visualization is also provided:



Figure 11.4: Non-Max Supression Vis

11.2.4 YOLO

The original authors of YOLO titled it as such for the reason that it only required a single pass on the image to accomplish the detection task. This is contrast to the other approaches used by Fast R-CNN and sliding window methods.

The output coordinates of the bounding box were detected using more straightforward regression techniques

11.2.4.1 YOLOv1

PASCAL-VOC AP: 63.4%

YOLOv1 predicted all bounding boxes simultaneously by the following process:

1. divide image into $S \times S$ grid
2. predict B bounding boxes of the same class and confidence for C different classes per grid element
3. each bounding box had five values:
 1. P_c - confidence score for the bounding box - how likely it contains an object and the accuracy of the box
 2. bx and by - coordinates of center of box relative to grid cell.
 3. bh and bw - height and width of box relative to full

4. output an $S \times S \times (B \times 5 + C)$ tensor
5. (optional) NMS used to remove redundant bounding boxes

Here is an example of that output:

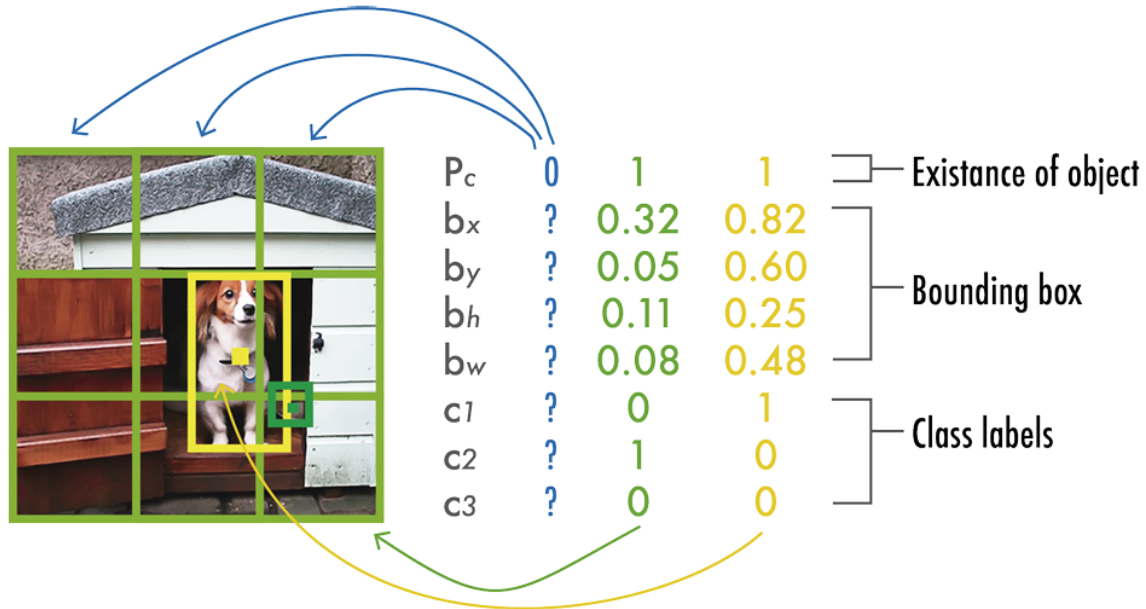


Figure 11.5: yolo output prediction

11.2.4.1.1 v1 Architecture

Normal Architecture

- 24 conv layers
 - 1×1 conv layers are used - reduce number of feature maps and keep parameters lower
 - leaky rectified linear unit activations
- 2 fc layers
 - predict bounding box coordinates / probs
 - linear activation function for final layer

FastYOLO

- Used 9 conv layers instead of 24 for greater speed (at the cost of reduced accuracy)

	Type	Filters	Size/Stride	Output
	Conv	64	$7 \times 7 / 2$	224×224
	Max Pool		$2 \times 2 / 2$	112×112
	Conv	192	$3 \times 3 / 1$	112×112
	Max Pool		$2 \times 2 / 2$	56×56
1×	Conv	128	$1 \times 1 / 1$	56×56
	Conv	256	$3 \times 3 / 1$	56×56
	Conv	256	$1 \times 1 / 1$	56×56
	Conv	512	$3 \times 3 / 1$	56×56
	Max Pool		$2 \times 2 / 2$	28×28
4×	Conv	256	$1 \times 1 / 1$	28×28
	Conv	512	$3 \times 3 / 1$	28×28
	Conv	512	$1 \times 1 / 1$	28×28
	Conv	1024	$3 \times 3 / 1$	28×28
	Max Pool		$2 \times 2 / 2$	14×14
2×	Conv	512	$1 \times 1 / 1$	14×14
	Conv	1024	$3 \times 3 / 1$	14×14
	Conv	1024	$3 \times 3 / 1$	14×14
	Conv	1024	$3 \times 3 / 2$	7×7
	Conv	1024	$3 \times 3 / 1$	7×7
	Conv	1024	$3 \times 3 / 1$	7×7
	FC		4096	4096
	Dropout 0.5			4096
	FC		$7 \times 7 \times 30$	$7 \times 7 \times 30$

Figure 11.6: yolo v1 architecture

11.2.4.1.2 v1 Training

Basic training process:

1. pretrain first 20 layers at resolution 224×224 with *ImageNet* dataset
2. add last four layers with randomly initialized weights - fine tune model with PASCAL VOC 2007 and PASCAL VOC 2012 at resolution 448×448

Loss functions:

- scaling factors
 - $\lambda_{coord} = 5$ - gives more weight to boxes with objects
 - $\lambda_{noobj} = 0.5$ - reduces importance of boxes with no object
- localization loss:
 - first two terms
 - computes error in predicted bounding box locations (x, y) and (w, h)
 - only penalizes boxes with objects in them
- confidence loss:
 - confidence error when object is detected (third term)
 - confidence error when no object is in box (fourth term)
- classification loss:
 - squared error of class conditional probabilities for each class if an object appears in the cell

11.2.4.2 YOLOv2 (YOLO 9000)

PASCAL-VOC AP: 78.6%

Improvements / Changes

1. Batch normalization - included on all convolutional layers
2. Higher resolution classifier - pretrained model (224×224) and then fine-tuned with ImageNet at a higher resolution (448×448) for ten epochs
3. fully convolutional - remove dense layers and use fully conv architecture
4. use anchor boxes to predict bounding boxes
 1. anchor box - box with predefined shapes for prototypical objects
 2. defined for each grid cell
 3. system predicts coordinates and class for every anchor box

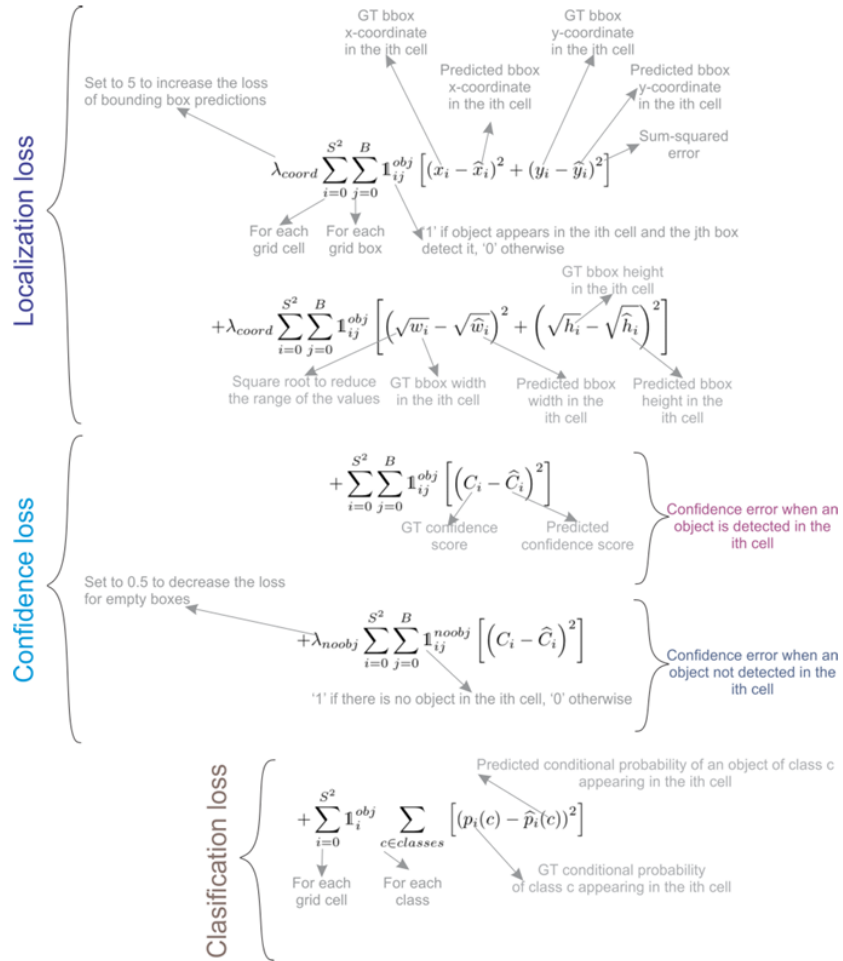


Figure 11.7: yolo v1 loss function

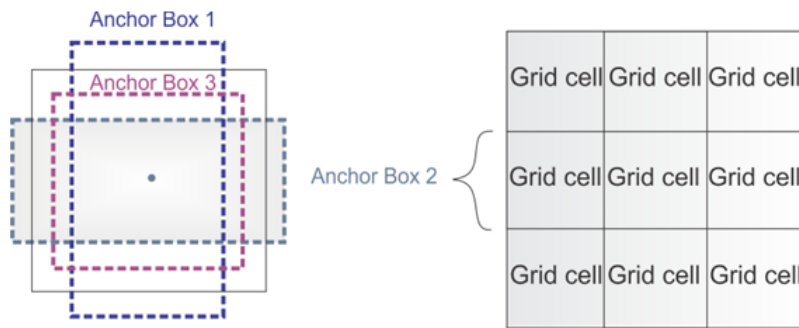


Figure 6: Anchor boxes. YOLOv2 defines multiple anchor boxes for each grid cell.

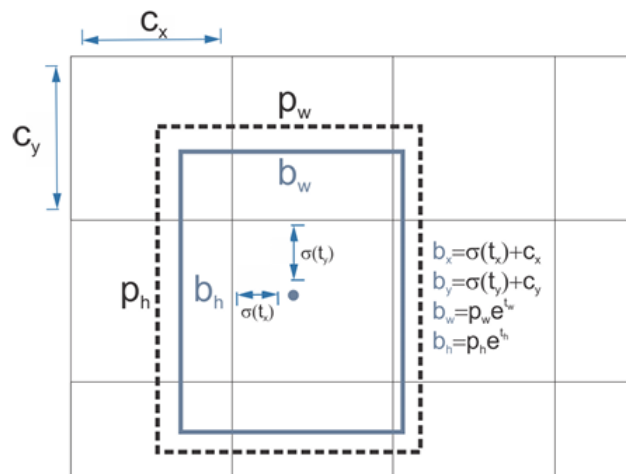


Figure 11.8: yolo v2 anchor boxes

5. Dimension clusters - pick good anchor boxes using k-means clustering on the training bounding boxes - improves accuracy of bounding boxes
6. Direct Location Prediction
7. Finer-grained features
 1. removed pooling layer - get feature 13 x 13 feature map for 416 x 416 images
 2. passthrough layer - 26 x 26 x 512 feature map -> stack adjacent features into different channels
8. Multi-scale training - train on different input sizes to make model robust to different input types

11.2.4.2.1 v2 Architecture

- backbone architecture -> Darknet-19
 - 19 conv layers
 - 5 max pool layers
 - * non-linear operation - uses OT to perform efficiently
 - use 1×1 conv between 3×3 to reduce parameters
 - batch normalization to help convergence
 - object classification head (replaces last 4 conv layers of YOLOv1)
 - * 1 conv layer (1000 filters)
 - * GAP layer
 - * Softmax classifier

11.2.4.3 YOLOv3

MS COCO AP: 36.2% AP(50): 60.6%

The code used to run YOLOv3 in Torch is provided at this [repository](#)

11.2.4.3.1 YOLOv3 Architecture

YOLOv3 makes use of a larger network architecture (backbone) called Darknet-53.

- replaces all max-pooling layers with **strided convolutions** and added residual connections (what are residual connections?) - see Chapter 10 for more information on this primitive.

The darknet architecture is presented here as well (visually):

Num	Type	Filters	Size/Stride	Output
1	Conv/BN	32	$3 \times 3 / 1$	$416 \times 416 \times 32$
2	Max Pool		$2 \times 2 / 2$	$208 \times 208 \times 32$
3	Conv/BN	64	$3 \times 3 / 1$	$208 \times 208 \times 64$
4	Max Pool		$2 \times 2 / 2$	$104 \times 104 \times 64$
5	Conv/BN	128	$3 \times 3 / 1$	$104 \times 104 \times 128$
6	Conv/BN	64	$1 \times 1 / 1$	$104 \times 104 \times 64$
7	Conv/BN	128	$3 \times 3 / 1$	$104 \times 104 \times 128$
8	Max Pool		$2 \times 2 / 2$	$52 \times 52 \times 128$
9	Conv/BN	256	$3 \times 3 / 1$	$52 \times 52 \times 256$
10	Conv/BN	128	$1 \times 1 / 1$	$52 \times 52 \times 128$
11	Conv/BN	256	$3 \times 3 / 1$	$52 \times 52 \times 256$
12	Max Pool		$2 \times 2 / 2$	$52 \times 52 \times 256$
13	Conv/BN	512	$3 \times 3 / 1$	$26 \times 26 \times 512$
14	Conv/BN	256	$1 \times 1 / 1$	$26 \times 26 \times 256$
15	Conv/BN	512	$3 \times 3 / 1$	$26 \times 26 \times 512$
16	Conv/BN	256	$1 \times 1 / 1$	$26 \times 26 \times 256$
17	Conv/BN	512	$3 \times 3 / 1$	$26 \times 26 \times 512$
18	Max Pool		$2 \times 2 / 2$	$13 \times 13 \times 512$
19	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
20	Conv/BN	512	$1 \times 1 / 1$	$13 \times 13 \times 512$
21	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
22	Conv/BN	512	$1 \times 1 / 1$	$13 \times 13 \times 512$
23	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
24	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
25	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
26	Reorg layer 17			$13 \times 13 \times 2048$
27	Concat 25 and 26			$13 \times 13 \times 3072$
28	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
29	Conv	125	$1 \times 1 / 1$	$13 \times 13 \times 125$

Figure 11.9: yolo v2 architecture

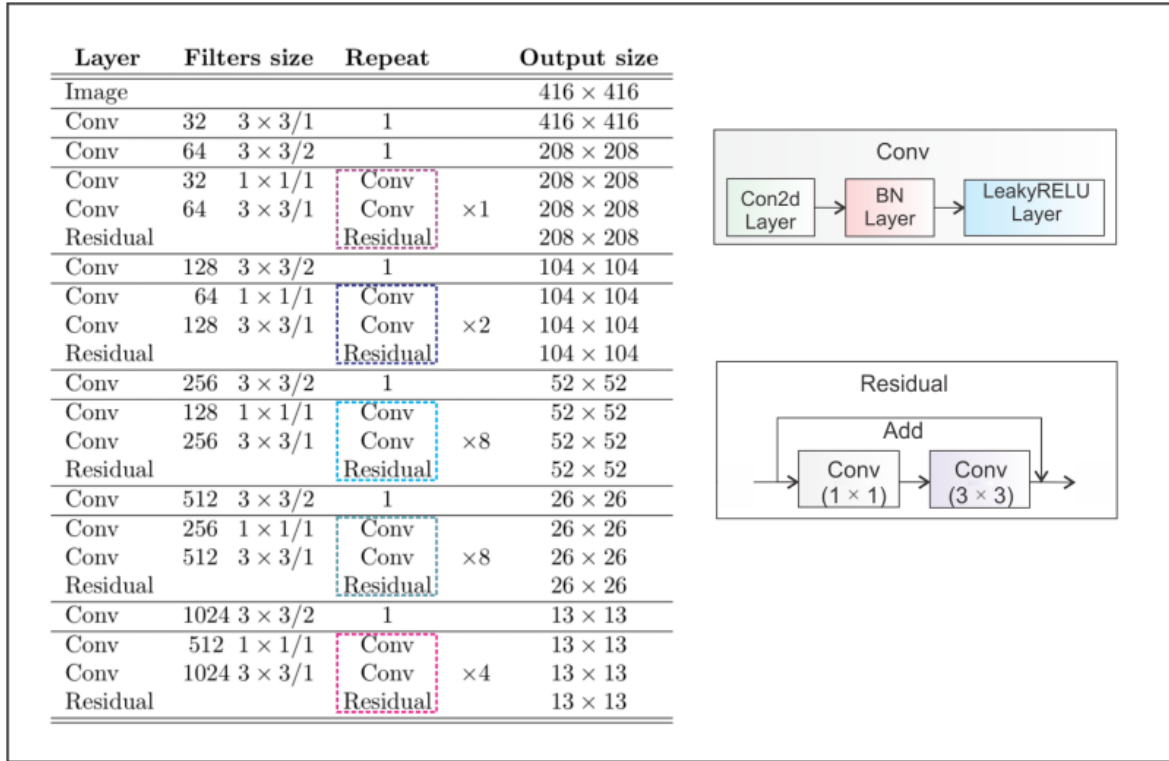


Figure 11.10: yolo v3 architecture

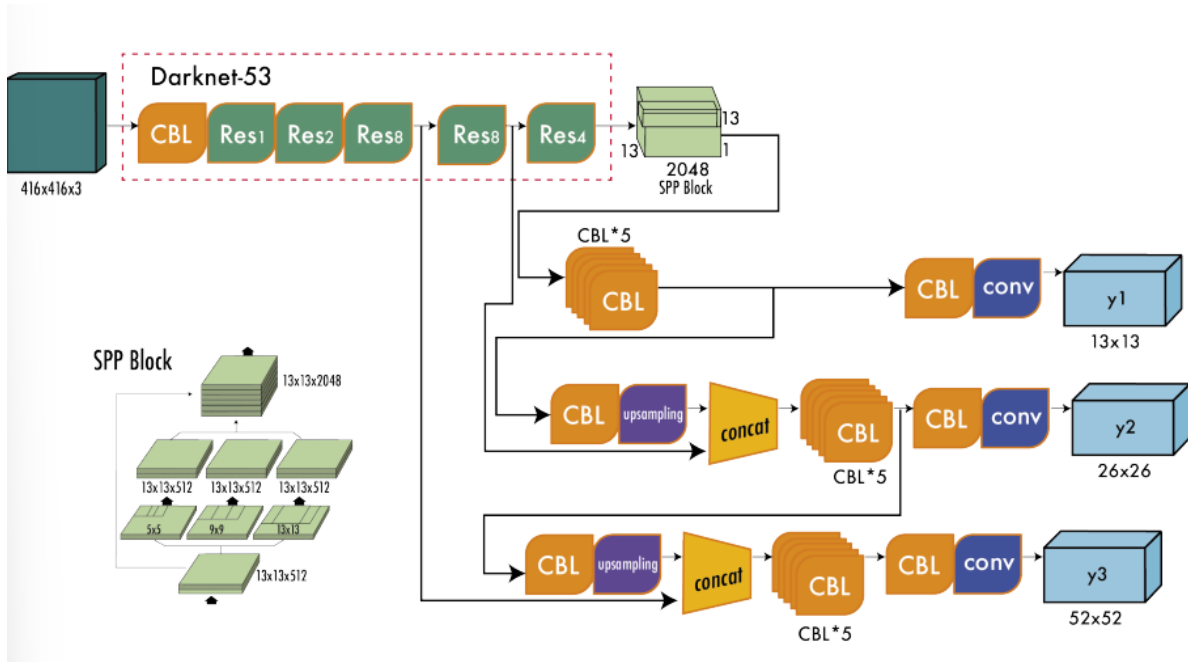


Figure 11.11: darknet 53 backbone

11.2.4.3.2 Multi-Scale predictions

- enables multi-scale predictions (predictions at multiple grid sizes)
- this helps to obtain finer detailed boxes (improves prediction of smaller boxes)
- YOLOv3 generates three separate outputs:
 - **y1**: 13×13 grid defines the output
 - **y2**: concatenating output after $(Res \times 4)$ with output of $(Res \times 8)$ - upsampling occurs from **y1** since the feature maps are of different sizes (13×13) and (26×26)
 - **y3**: upsample **y2** output to match 52×52 feature maps

11.2.4.3.3 Backbone, Neck, and Head

After release of YOLOv3, object detectors began to be described in terms of the backbone, neck, and head

Backbone

- Extracts useful features from the input image.
- A convolutional neural network trained on large-scale image classifications task (ImageNet)
- captures hierarchical features at different scales

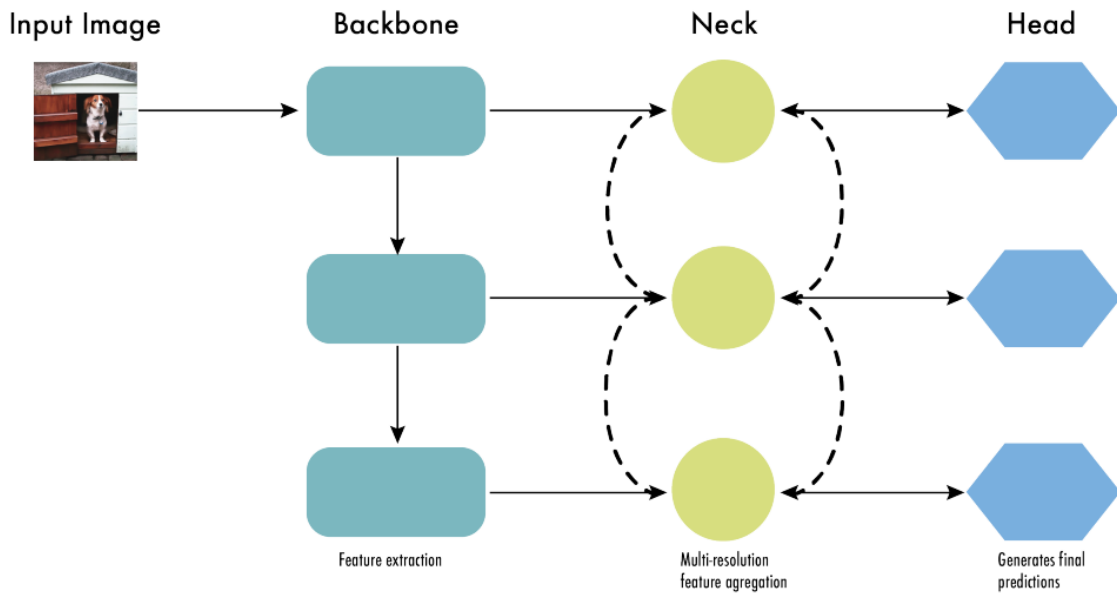


Figure 11.12: modern object detection architecture

- low-level features - earlier layers
- high-level features - deeper layers

Neck

- aggregates / refines features extracted by backbone
 - enhance spatial / semantic information across different scales
 - includes conv layers
 - includes feature pyramid networks

Head

- makes predictions based on features provided by backbone and neck
- consists of task-specific subnetworks to perform *classification*, *localization*, *localization*, *instance segmentation* and *pose estimation*
- non-maximum suppression filters out overlapping predictions (retains only most confident detections)

11.2.4.4 YOLOv4

MS COCO AP: 43.5% AP(50): 65.7%

The philosophy of YOLOv4 approaches optimization of the model into two categories: *bag of freebies* and *bag of specials*

Bag of Freebies:

- increase training time / cost
- do not affect inference time
- examples include *data augmentation*

Bag of Specials:

- increase inference time / cost
- improve accuracy of the model (MaP)
- examples include
 - enlarging receptive field ???
 - combining features ???
 - post-processing ???

11.2.4.4.1 Model Improvements

- **Enhanced network architecture with Bag-of-Specials**

- backbone: **Darknet-53** + Cross-stage Partial Connections (CSPNet) ?? + Mish Activation Function ??
 - * *tested several backbone architectures to choose best option*
 - * CSP reduces computation while maintaining accuracy
- neck: Spatial Pyramid Pooling (SPP)?? + **Multi-scale predictions** + modified path aggregation network PANet + modified spatial attention module (SAM)
 - * SPP increases receptive field without affecting inference speed
- detection head: **Same anchors as YOLOv3**

- **Advanced training with Bag-of-Freebies**

- regular augmentation
 - * random brightness, contrast, scaling, cropping, flipping, rotation
- special
 - * **mosaic augmentation:**
 - combines four images into a single one
 - reduces need for large mini-batches for batch normalization
 - * **DropBlock** regularization (instead of *Dropout*)
 - * **CIoU** loss and Cross mini-batch normalization **CmBN** for collecting statistics from the entire batch instead of just mini-batches

- these changes improve the **detector**
- **Self-adversarial Training**
 - improves model robustness to perturbations
- **Hyperparameter optimization with Genetic Algorithms**
 - genetic algs used on first 10% of periods
 - cosine annealing scheduler to alter learning rate during training

11.2.4.4.2 YOLOv4 Architecture

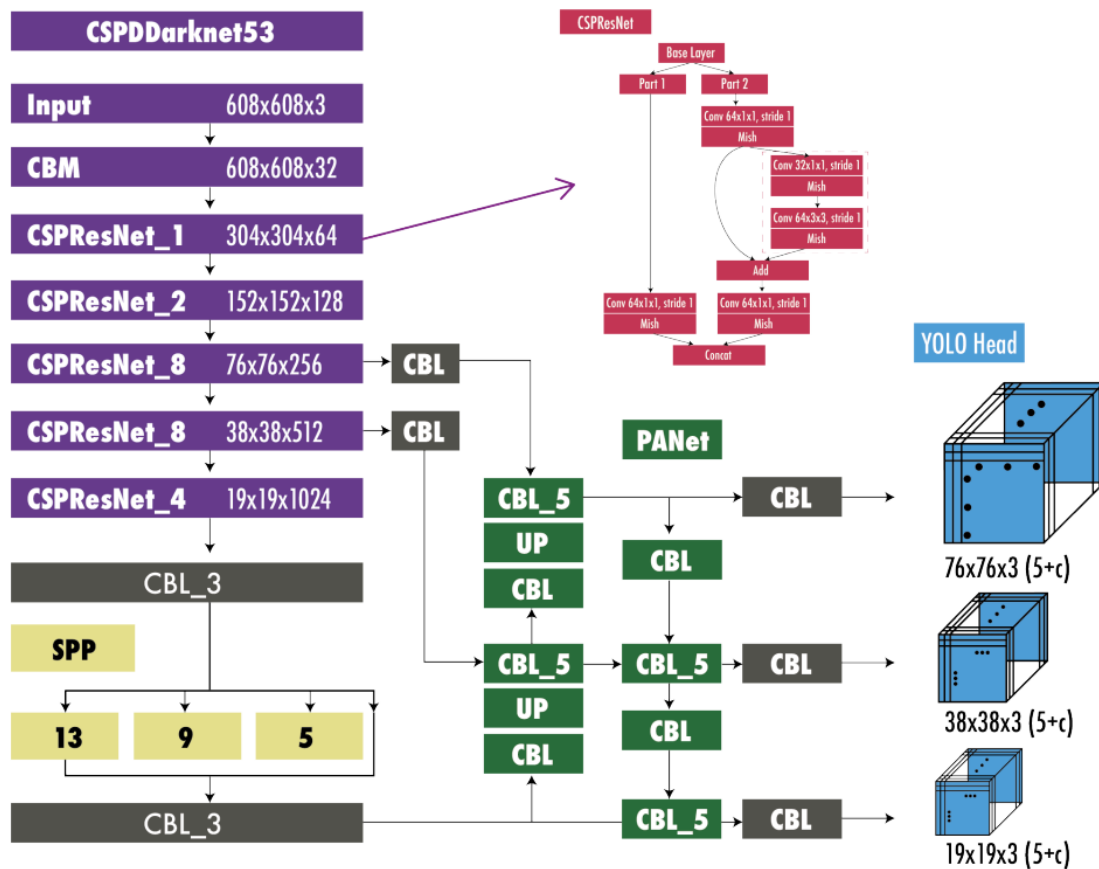


Figure 11.13: yolo v4 architecture

11.2.4.5 YOLOv5

YOLOv5x MS COCO AP: 50.7% AP(50): X%

No paper exists for YOLOv5 but there are several key advantages to using it

- It is developed in PyTorch instead of Darknet
- is open source and actively maintained by ultralytics
- is easy to use, train, and deploy
- many integrations for labeling, training, and deployment (mobile)

There are several scaled versions of this model:

- **YOLOv5n** (nano)
- **YOLOv5s** (small)
- **YOLOv5m** (medium)
- **YOLOv5l** (large)
- **YOLOv5x** (extra large)

This is useful for V2X privacy preserving applications because we will want to make use of smaller networks than might otherwise be used and compare their performance (efficiency) against larger networks.

This helps to answer the question of whether the accuracy tradeoff is worth the speedup.

11.2.4.5.1 Results

YOLOv5x achieves the following results

- AP 50.7% on COCO [batch size = 32, 640 pixels]
- AP 55.8% on COCO [1536 pixels]

11.2.4.6 Scaled-YOLOv4

YOLOv4-large MS COCO AP: 55.6%

Utilized scaling techniques (making larger to increase accuracy at the expense of speed, and scaling down increases speed at the expense of accuracy). Scaled-down models require less compute power and can run on embedded systems

Like YOLOv5, was developed in **PyTorch** instead of **Darknet**

11.2.4.6.1 Results

- YOLOv4-large - MS COCO AP: 56%
- YOLOv4-tiny - MS COCO AP: 22%

11.2.4.7 YOLOR

MS COCO AP: 55.4% AP(50): 73.3%

YOLOR stands for “you only learn one representation” and is novel for introducing multi-task learning approach which creates a single model for multiple tasks (classification, detection, pose estimation) by learning a general representation and using sub-networks to create task-specific representations!

11.2.4.7.1 Results

- YOLOR - MS COCO AP: 55.4% AP(50): 73.3%

11.2.4.8 YOLOX

MS COCO AP: 50.1%

Was designed off the back of the Ultralytics YOLOv3 (see Section 11.2.4.3) in PyTorch.

11.2.4.8.1 Model Changes and Improvements

- Anchor Free - simplified training and decoding process
- Multi Positives - center sampling (assigned center 3×3 area as positives) to account for imbalances produced by lack of anchors
- Decoupled Head - classification confidence and localization accuracy separated into two heads (connecting them leads to some misalignment)
 - **sped up model convergence and improved accuracy**
- Advanced Label Assignment - ambiguities were associated with ground truth bounding boxes (box overlap) - this is addressed with assigning procedure as Optimal Transport Problem . The authors develop a simplified version called simOTA
- Strong augmentations - MixUP and Mosaic augmentations were used

11.2.4.8.2 Architecture

11.2.4.9 YOLOv6

MS COCO AP: 52.5% AP(50): 70%

Adopted an anchor-free detector and provided a series of models at different scales for nuanced applications.

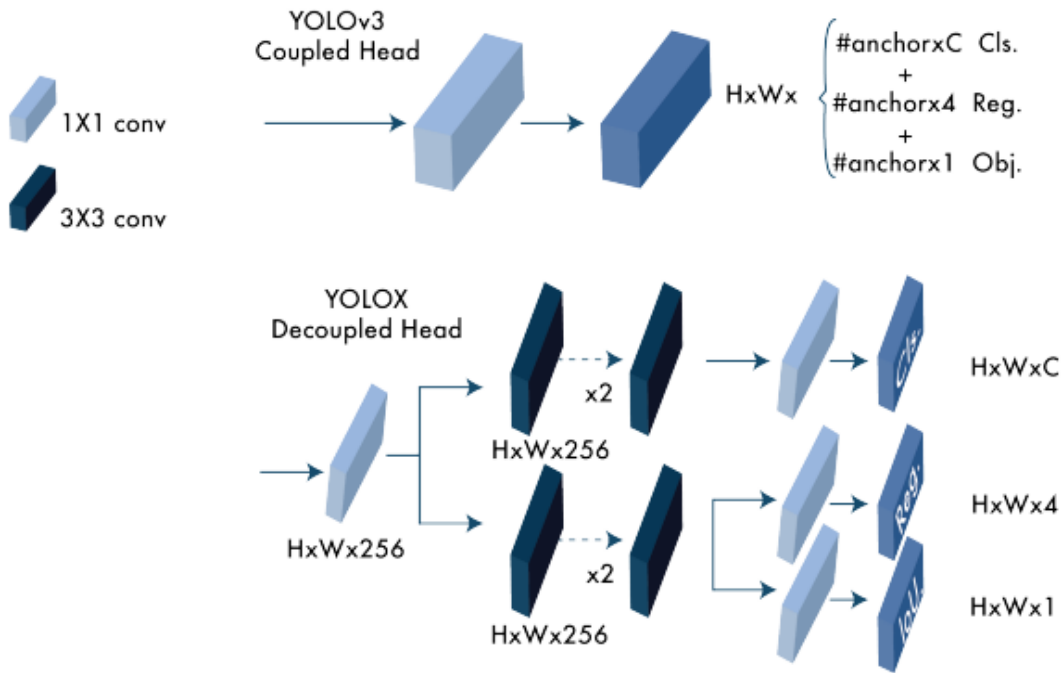


Figure 11.14: yolox architecture

11.2.4.9.1 Model Changes and Improvements

- New architecture backbone based on RepVGG . higher parallelism and use of neck based on RepBlocks or CSPStackRep Blocks and developed an efficient decoupled head
- Label Assignment with TOOD
- New classification and regression losses using VariFocal loss and SIOU / GioU regression loss
- self-distillation for regression and classification tasks
- quantization scheme for detection with RepOptimizer channel-wise distillation for a faster detector

11.2.4.10 YOLOv7

MS COCO AP: 55.9% AP(50): 73.5% - input 1280 pixels

Compared to YOLOv4 and YOLOR, YOLOv7 achieves a significant reduction in parameters used, and improves average precision by a meaningful increase in average precision as well.

11.2.4.10.1 Model Changes and Improvements

Architectural Changes

- Extended efficient layer aggregation network - ELAN allows for more efficient training and convergence
- Model Scaling for concatenation-based models - maintain optimal structure of the model by scaling the depth and width of the block with the same factor

Bag-of-Freebies

- Planned re-parameterized convolution - Architecture is inspired by RepConv. Identity connection outperforms residual in Resnet (Chapter 10) and concatenation in DenseNet
- Coarse label assignment for auxiliary head (training) and fine label assignment for the lead head (final out)
- Batch normalization in conv-bn-activation - integrates mean and variance of batch normalization into weight and bias of convolutional layer at inference stage (batch norm folding)
- Implicit knowledge (see Section 11.2.4.10)
- Exponential moving average as final inference model

11.2.4.11 DAMO-YOLO

MS COCO AP: 50.0%

Was inspired by a series of technologies relevant at the time and provided tiny/small/medium scaled model variants

- Neural architecture search (this was also used in Delphi) - MAE NAS -
- a large neck
- a small head
- aligned OTA label assignment - uses focal loss for classification cost and IoU of prediction / ground truth box as *soft label*. Enables selection of *aligned samples* for targets
- knowledge distillation
 - (1) teacher guiding student (stage 1)
 - (2) student fine-tuning (stage 2)
 - enhancements in distillation approach
 - * align module - adapts student features to same resolution as teacher's
 - * channel-wise dynamic temperature - normalizes teacher and student features

11.2.4.12 YOLOv8

YOLOv8x MS COCO AP: 53.9% (640 pixels)

A version of YOLO released by ultralytics which is anchor free and uses mosaic augmentation for training up to the last ten epochs. It provides five scaled versions

1. YOLOv8n (nano)
2. YOLOv8s (small)
3. YOLOv8m (medium)
4. YOLOv8l (large)
5. YOLOv8x (extra large)

11.2.5 PP-YOLO Models

The PP-YOLO models were developed in parallel with the standard YOLO variants. These models were based on YOLOv3 but were developed in the PaddlePaddle deep learning platform instead. The goal of their work was to show how an object detector should be constructed step-by-step, not to provide any novel functionality or a new approach.

11.2.5.1 PP-YOLO

MS COCO AP:45.9% AP(50): 65.2%

11.2.5.1.1 Divergence from YOLOv3

- ResNet50-vd backbone + deformable convolutions replace DarkNet-53 (Section [11.2.4.3](#))
 - achieves higher classification accuracy on Imagenet
- Larger batch size - improves training stability
- Maintained moving averages for trained parameters
- DropBlock applied on FPN
- IoU loss added with L1-Loss for bounding box regression
- IoU prediction branch for measuring localization accuracy (and optimization)
- Grid Sensitive Approach like YOLOv4 (Section [11.2.4.4](#)) - improves bounding box center prediction at grid boundary
- Matrix NMS (parallelized NMS for faster computation)
- CoordConv - 1×1 convolution of the FPN. Allows for learning translational invariance
- Spatial Pyramid Pooling increases receptive field of the backbone

11.2.5.1.2 PP-YOLO Augmentations and Preprocessing

- Mixup Training - weights $\sim \text{Beta}(\alpha = 1.5, \beta = 1.5)$
- Random color distortion
- Random expand
- Random crop and random flip with probability 0.5
- RGB channel z-score normalization $\mu = [0.485, 0.456, 0.406]$ and $\sigma = [0.229, 0.224, 0.225]$

11.2.5.2 PP-YOLOv2

MS COCO AP: 49.5%

11.2.5.2.1 Changes and Improvements

- Backbone changed from ResNet50 to ResNet101
- Path aggregation network instead of FPN
- Mish Activation function applied to the detection neck
- Larger input sizes - increases performance on small objects
- Modified IoU aware branch - soft label format instead of soft weight format

11.2.5.3 PP-YOLOE

MS COCO AP: 51.4% (78.1 FPS NVIDIA V100)

11.2.5.3.1 Changes and Improvements

- Anchor Free (following trends of other yolo models)
- New backbone and neck - modified neck with RepResBlocks (combine dense and residual connections)
- task alignment learning (see Section 11.2.4.8)
- efficient task-aligned head (ET-head) - single head based on TOOD instead of splitting the classification / detection heads like with YOLOX (Section 11.2.4.8)
- varifocal (VFL) and distribution focal loss (DFL)
 - VFL weights positive samples using a target score which places a greater importance on high-quality samples during training
 - DFL extends Focal Loss from discrete to continuous labels - better for representations which combine quality estimation and class prediction - this allows for better depiction of flexible distributions in real data (eliminates risk of inconsistency)

The authors provide several scaled models

- PP-YOLOE-s (small)
- PP-YOLOE-m (medium)
- PP-YOLOE-1 (large)
- PP-YOLOE-x (extra large)

11.2.6 Summary of YOLO

Table 4: Summary of YOLO architectures. The metric reported for YOLO and YOLOv2 were on VOC2007, while the rest are reported on COCO2017

Version	Date	Anchor	Framework	Backbone	AP (%)
YOLO	2015	No	Darknet	Darknet24	63.4
YOLOv2	2016	Yes	Darknet	Darknet24	63.4
YOLOv3	2018	Yes	Darknet	Darknet53	36.2
YOLOv4	2020	Yes	Darknet	CSPDarknet53	43.5
YOLOv5	2020	Yes	Pytorch	Modified CSP v7	55.8
PP-YOLO	2020	Yes	PaddlePaddle	ResNet50-vd	45.9
Scaled-YOLOv4	2021	Yes	Pytorch	CSPDarknet	56.0
PP-YOLOv2	2021	Yes	PaddlePaddle	ResNet101-vd	50.3
YOLOR	2021	Yes	Pytorch	CSPDarknet	55.4
YOLOX	2021	No	Pytorch	Modified CSP v5	51.2
PP-YOLOE	2022	No	PaddlePaddle	CSPRepResNet	54.7
YOLOv6	2022	No	Pytorch	EfficientRep	52.5
YOLOv7	2022	No	Pytorch	RepConvN	56.8
DAMO-YOLO	2022	No	Pytorch	MAE-NAS	50.0
YOLOv8	2023	No	Pytorch	YOLO v8	53.9

Figure 11.15: summary of yolo architectures and results

12 Implementation Guides for Yolo Object Detectors

Yolo5 is an extremely popular choice of object detector model. It has empirically demonstrated a high mean average precision, is efficient, supports an array of model scales, and is very easy to implement. Moreover, it is a torch port of the yolo family, which makes it even more popular among machine learning researchers and the like.

This guide provides a simple explanation for implementing the YOLOv5 model on a very simple example. Please note that this tutorial borrows heavily from the article written by Maindola (2021) . Practically speaking, the purpose of this entry is for ease of use in connecting to other useful references throughout the remainder of this book.

12.1 YOLOv5 Architecture In Depth

A comprehensive description of the YOLOv5 network architecture is provided at the [documentation source from ultralytics](#).

[Open Genus](#) also provides some good reference material on the architecture of the Yolo5 architecture.

This [youtube video](#) also provides a useful explanation for the partial connections structure of the YOLO model architecture.

Tensors are either added or concatenated together when they are passed through the network. YOLO makes exclusive use of tensor concatenation.

13 AdderNet

What is AdderNet? AdderNet is a novel approach to deep learning, specifically with convolutional neural networks which seeks to reduce computational complexity by replacing multiplication operations with additions. Namely, this is accomplished by supplanting convolution operations (cross-correlation) with l_1 -norm distance. This substitution can be made specifically because the cross-correlation is used to measure the similarity between the input feature and convolution filters. Thus, absolute value can also be used as such a measure.

Optimizations for the training of such adder networks have also been developed. Such improvements include enhanced back-propagation schemes with regularized gradients. This helps to ensure sufficient updates of templates and better network convergence

13.1 Defining Adder-Networks

Suppose the following setting:

- You have some filter $F \in \mathbb{R}^{d \times d \times c_{in} \times c_{out}}$
 - d = kernel size
 - c_{in} = input channel
 - c_{out} = output channel
- Input feature: $X \in \mathbb{R}^{H \times W \times c_{in}}$
- Output $Y(m, n, t) = \sum_{i=0}^d \sum_{j=0}^d \sum_{k=0}^{c_{in}} S(X(m+i, n+j, k), F(i, j, k, t))$

Here, S is the similarity measure. Adder-Networks use the l_1 distance as the measure of similarity between two points. Calculate the sum of the absolute differences of two points' vector representations (this does not contain any multiplications)

Formulation

The following presents a general form for a convolution-equivalent layer. Here $S(\cdot, \cdot)$ is a pre-defined similarity measure. When $S(x, y) = x \times y$, this is simply the convolution operation

$$Y(m, n, t) = \sum_{i=0}^d \sum_{j=0}^d \sum_{k=0}^{c_{in}} S(X(m+i, n+j, k), F(i, j, k, t)) \quad (13.1)$$

The absolute value (l1-norm) replaces the convolution operation (cross-correlation) as the measure of similarity between input features and the filter

$$Y(m, n, t) = - \sum_{i=0}^d \sum_{j=0}^d \sum_{k=0}^{c_{in}} |X(m+i, n+j, k) - F(i, j, k, t)| \quad (13.2)$$

13.1.1 Convolution v. L1-Distance

Differences are as follows:

- the convolution filter can exist as a weighted sum of values in the feature maps (can take a positive or negative value)
- adder filters are ALWAYS NEGATIVE
 - use batch normalization for help (normalize adder outputs to correct range)

Part III

V2X Applications

This portion of the text deals specifically with discussions of V2X applications and papers which have tackled these sorts of the problems in the past.

14 Red Light Violation Detection

14.1 Motivation

Secure Red Light Violation detection is an important application of secure machine learning protocols. Oftentimes, these systems will require the use of image segmentation and object recognition protocols. The images used for this task expose often-times sensitive data about individual users.

In the practical setting of interest, this means exposing license-plate information, associated vehicles, and location information available from the images themselves.

14.2 V2I Algorithms for RLR Detection

The authors of this paper have constructed V2I mechanisms for red light running (RLR) detection, wrong way entry (WWE), and an array of other import tasks in the context of V2X. See the citation Dokur and Katkoori (2022)

14.2.1 Red Light Violation Detection Algorithm

The proposed system utilizes the following logic to detect whether a car will violate a red light. A car which is approaching an intersection is connected to road-side units (RSUs) which are installed at traffic lights in an intersection.

Each light is said to be located at points $B(x_2, y_2, z_2)$, $C(x_3, y_3, z_3)$, $D(x_4, y_4, z_4)$ and $E(x_5, y_5, z_5)$ respectively.

Unlike image-based systems, this system assumes V2I communication between the traffic lights and the vehicle in question. This means that the traffic state does not need to be determined by an image classifier. Rather, we already have this information by default.

14.3 Thao et.al on Traffic Violation Detection

The paper proposed by L. Thao (2022) introduces a mechanism for detecting red light violations automatically. There paper is titled: *Automatic Traffic Red-Light Violation Detection Using AI*

14.3.1 Problem Setting

The reason AI technologies (image classification and detection) systems are better suited than standard sensor technologies is that they are able to operate more consistently, even when the number of vehicles in the setting increases dramatically.

14.3.2 System Design and Solution Approach

Separate the task into three parts:

1. vehicle violation detection
2. red signal change monitoring
3. vehicle recognition

Vehicle Violation Detection

The YOLOv5s pretrained model (COCO dataset) is used for detecting violating vehicles. After detecting violation, following frames are used to try and determine the license plate (identify vehicle). See Section 11.2.4.1 for more information on the YOLO object detection model.

Below, a picture of the overall system flow is presented:

- **vehicle tracking** - performed every 5 frames
 - if IOU (intersection over union) of bounding box is close to one from a previous frame, then the car is assumed to be the same one from that frame.
- **violation line detection**
 - image processing is used to determine traffic lines
 - boundary lines are drawn onto frames captured by the camera later
- **traffic state detection**
 - color filters and image processing used to detect changes in the state of the traffic light

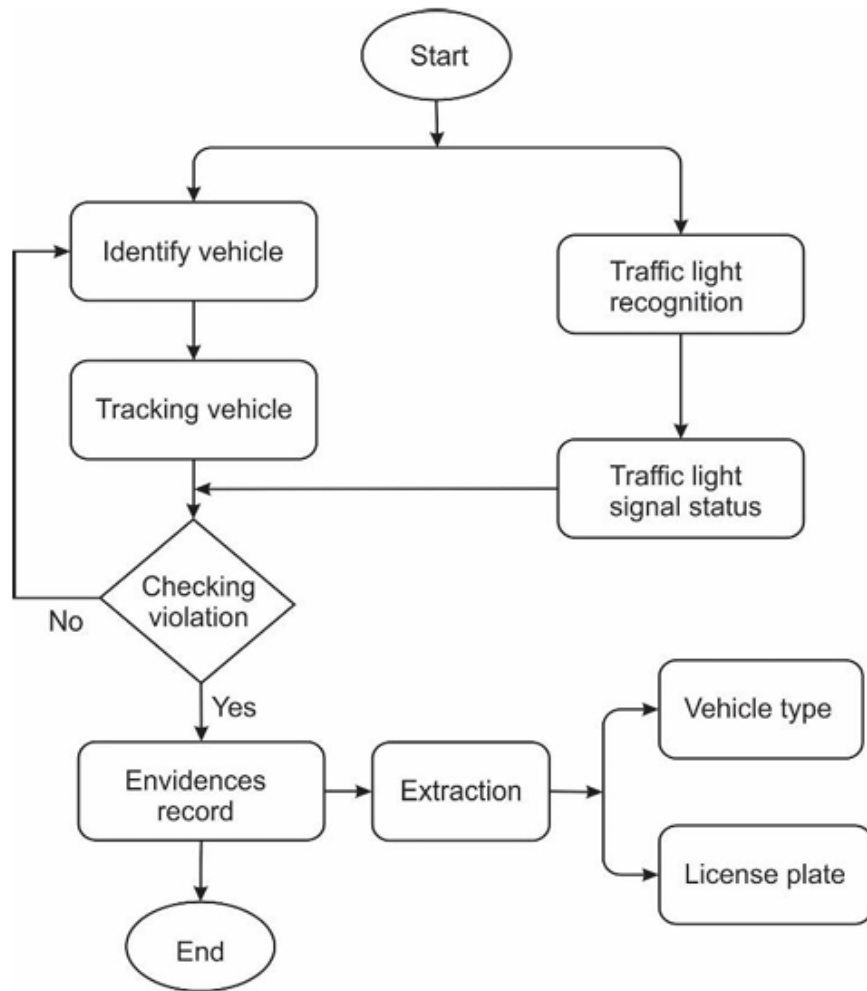


Figure 2. Algorithm of violation detection

Figure 14.1: system flow

14.3.3 Primary Contributions

1. Implementation of modified *YOLOv5s* model
 1. used parameter changes from original model
 2. achieved following accuracy results:
 1. 82% - vehicle identification
 2. 90% - traffic signal status change
 3. 86% - violation detection
2. Best Performing Architecture given below (v3 / v4)

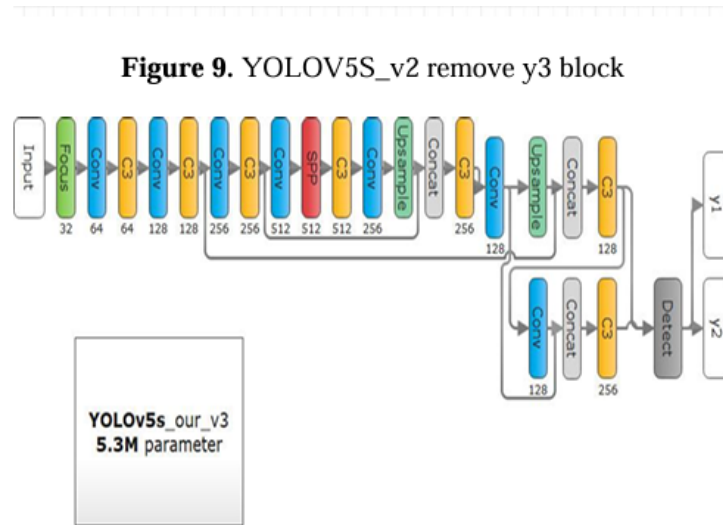


Figure 10. YOLOV5S_v3 reduce ½ filter in Conv, remove y3 block

Figure 14.2: modified Yolo architecture

According to a labmate, it may only be necessary to fine tune the YOLO model on 10 epochs. It may not be necessary to do any more than that.

14.4 Goma et.al on RLR Detection with SSD (Single Shot Detector)

Work by J. Goma (2020) demonstrates the ability to detect red-light-running and over-speeding with a high level of accuracy. Specifically, they achieve **100%** accuracy on red light running detection and **92.1%** accuracy for over-speeding viola-

tions. They accomplish these results using a CNN applied to an SSD (single deep neural network)

14.4.1 Methods and System Design

15 Traffic Flow Forecasting

15.1 Spatio-Temporal Graph Convolutional Networks (STGCN)

These notes refer to the conference publication B. Yu (2018) code implementation is provided at github.com

15.1.1 Graph Convolutions

Similar to ASTGCN presented in Section 15.2, STGCN utilizes graph convolutions and temporal convolutions in order to compute spatial correlations in the graph data representations.

The graph convolution operator, “ $\ast_g$ ” is used for identifying the spatial correlations in the data:

$$\Theta \ast_g x = \Theta(L)x = \Theta(U\Delta U^T)x = U\Theta(\Delta)U^T x$$

Again, the graph Fourier basis $U \in \mathbb{R}^{n \times n}$ is used in order to transform the computation space of the convolution to the spectral space. This Fourier basis is the matrix of eigenvectors of the *normalized graph Laplacian* L . See [this lecture](#) on the normalized Laplacian matrix for more information ...

$$L = I_n - D^{-\frac{1}{2}} W D^{-\frac{1}{2}}$$

15.1.1.1 The Normalized Laplacian

[The normalized laplacian](#)

Important Definitions:

- multiset: a generalization of the concept of a set. Unlike a set which allows for only one instance of each of its elements, a multiset may contain multiple instances of the same element. The number of times a given element appears in the multiset is known as its *multiplicity*. As with normal sets, the order of elements is not relevant.

- cardinality: the sum of multiplicities for all elements (number of items)

Spectral Graph Theory

- What are the relationships between the structure of a given graph and the eigenvalues of a matrix associated with the graph?
 - Adjacency matrix: indicates which vertices are adjacent. Eigenvalues can count closed walks, so can count edges and test bipartite-ness
 - Laplacian $L = D - A$: positive semi-definite. Eigenvalues can count edges and number of components
 - Signless Laplacian $Q = D + A$: unsigned incidence matrix, positive semi-definite. Eigenvalues can count edges and number of bipartite components
 - **Normalized Laplacian** $\mathcal{L} = D^{-1/2}(D-A)D^{-1/2}$: normalizes the Laplacian matrix, and is tied to the probability transition matrix
 - * eigenvalues $\in [0, 2]$
 - * multiplicity of 0 is number of components
 - * multiplicity of 2 is number of bipartite components
 - * tests for bipartite-ness
 - * cannot always detect number of edges

Normalizing matrices is useful.

15.1.2 Network Architecture

Components:

- two spatio-temporal convolutional blocks (ST-Conv Blocks) and FC output layer $\rightarrow \hat{V}$
 - residual connections and bottlenecks are applied within each block
 - ST-Conv Blocks contain
 - * *one* spatial graph convolution layer (in the middle)
 - * *two* temporal gated convolution layers

Graph CNNs for Extracting Spatial Features

Because the kernel computation Θ in the graph convolution can be expensive due to $O(n^2)$ multiplications with graph Fourier basis, approximation strategies are utilized:

1. Chebyshev polynomial approximation
2. 1st-order approximation

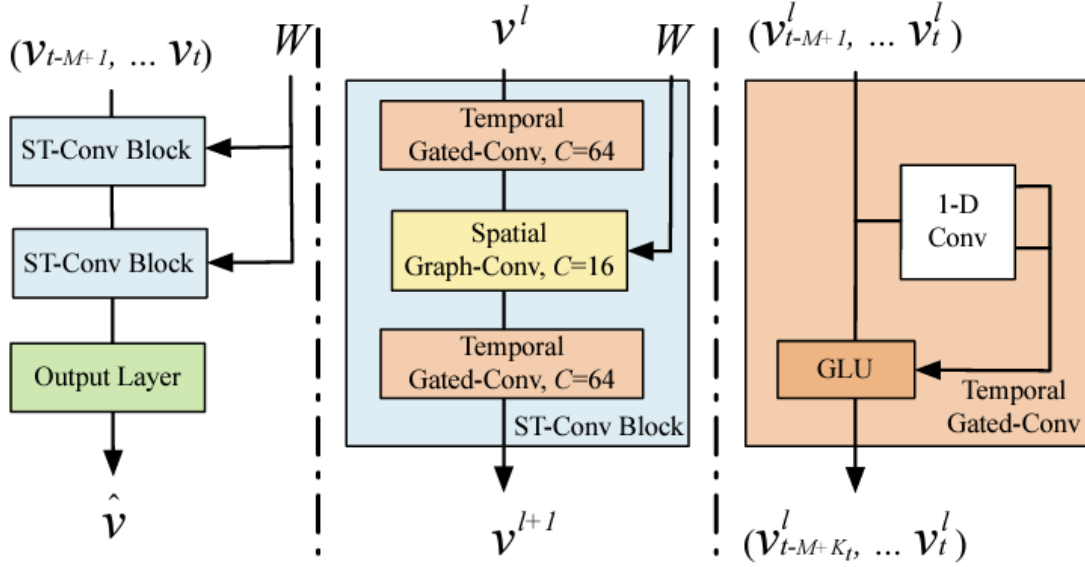


Figure 15.1: STGCN Network Architecture Diagram

15.1.3 Graph Convolution Approximations

Chebyshev Polynomials Approximation

- chebyshev polynomials are used in order to reduce the numebr of parameters required by the kernel Θ .
- Chebyshev polynomials usually approximate kernels as truncated expansion of order $K - 1$

$$\Theta(\Delta) = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{\Delta})$$

- $\tilde{\Delta} = 2\Delta/\lambda_{max} - I_n$
- λ_{max} is the greatest eigenvalue of the Laplacian matrix

- Mathematical formulation for the graph convolution:

$$\Theta *_g x \sim \sum_{k=0}^{K-1} \theta_k T_k(\tilde{L})x$$

- $T_k(\tilde{L}) \in \mathbb{R}^{n \times n}$ - k th order Chebyshev polynomial, evaluated at the scaled laplacian, $\tilde{L}$

First Order Chebyshev Approximations

- first order approximation is used
- allows for a deeper architecture to be used
 - not restricted to a specific parameterization by the polynomials
- scaling / normalization in Neural Networks ensure $\lambda_{max} \sim 2$
- assumptions simplify the graph convolution operation:

$$\Theta *_g x \sim \theta_0 x - \theta_1 (D^{-1/2} W D^{-1/2}) x$$

- θ_0 and θ_1 - shared parameters of the kernel
- W and D are also renormalized
- What does the D matrix indicate?

15.1.4 Gated CNNs - temporal feature extraction

- prior art uses RNNs (RNN-based models)
- RNNs have several critical limitations:
 - time-consuming iterations
 - slow response to dynamic changes
 - complex gate mechanisms
- convolutional structures employed in entirety on time axis to capture temporal behaviors of traffic flows
 - convolutions are faster for training
 - allows for forming hierarchical representations
- each node of the graph, g , temporal convolutions explore K_t neighbors without padding
 - reduces length of sequences by $K_t - 1$ each time
 - * I don't have any idea what this means though ...
 - inputs to temporal convolutions - length- m sequences with C_i channels, $Y \in \mathbb{R}^{m \times C_i}$
- Definition of gated temporal convolution:

$$\Gamma *_\tau Y = P \odot \sigma(Q) \in \mathbb{R}^{M-K_t+1} \times C_0$$

- is generalizable to the 3D convolution

15.1.5 Spatio-Temporal Convolutional Block

- (spatial) graph convolutions are combined with gated (temporal) convolutions
 - temporal -> spatial -> temporal

- residual connections applied for improved training / avoid overfitting
- layer normalization used to prevent overfitting too
- General expression:

$$v^{l+1} = \Gamma_1^l *_{\tau} \text{ReLU}(\Theta^l *_{\mathcal{g}} (\Gamma_0^l *_{\tau} v^l))$$
 - Γ_0^l - upper temporal kernel in block l
 - Γ_1^l - lower temporal kernel in block l
 - Θ^l - spectral kernel of graph convolution
- full structure:
 - 2 ST-Conv Blocks
 - 1 Temporal Convolution
 - Fully connected layer to outputs

15.1.6 Prediction Task

- goal - predict speeds of cars for each of n nodes
- loss - L2 loss function

15.1.7 Experimental Evaluation / Study

Datasets

datasets contain key info of traffic observations / geographic information with corresponding timestamps

- BJER4 - Beijing Municipal Traffic Commission
- PeMSD7 - California Department of Transportation

Data Cleaning

- linear interpolation used for imputation
- data inputs normalized (z-score method)

Data Formatting

- separate methods are utilized in order to convert traffic data into the proper format for graph network analysis.

15.1.8 Topic Study Shortlist

- gated sequential convolutions
 - what does it mean for these convs. to be gated? [DONE]
- what are chebyshev polynomials and how are they used to approximate convolutions?
- what are first-order approximations of the graph laplacian (see page 3 of STGCN paper)
 - kipf and welling 2016
- what does it mean for graph convolutions to be stacked vertically as opposed to being computed horizontally like traditional convolution operations
- what are gated linear units? (GLU) for non-linearity
- what is linear interpolation imputation?

actually, understanding the theory behind these mechanisms is not especially relevant for my work. I simply need to know what the approximation is that they used for their computations. i.e. - we need to know the results, not how they were obtained.

15.1.9 What are Gated Convolutions?

Notes based on article from serp.ai

- Gated Convolutions - specific type of convolution that include a *gating mechanism*
- a sigmoid activation function determines which part of the input should be passed to the output and which parts should be “gated” or suppressed

Process

- input is convolved with a filter (like a regular convolution)
- before the result is generated, the input (after convolving) is passed through the gating mechanism
 - sigmoid activation determines the part of the input are passed to the output, and which are suppressed (hence gated)
 - use zero-padding - ensure output only depends on past inputs and doesn't depend upon future inputs.
- this strategy might improve interpretability - it may be easier to understand why certain predictions are made based on the results of the gating mechanism
- no activation is further applied after the gating mechanism has run

Gated Convolutions in STGCN

- Gated convolutions used in the Temporal Convolution Layer

- when the activation function (gating mechanism) is the “GLU”, the following formula is used: $(x_{\{p\}} + x_{\{in\}}) \cdot (x_{\{q\}})$

15.1.10 Approximated Convolutions (Chebyshev Polynomial Approximations)

Notes based on the youtube video [Convolutions and Polynomial Approximation](#) by kieransquared

- Approximations - finding a sequence of polynomials, P_1, P_2, P_3 such that P_n converges to f
- [An introduction to spectral graph theory](#)
- spectral graph theory is based on the idea that specific properties of a given graph can be extracted from the eigenvalues, or approximate eigenvalues of a given adjacency or laplacian matrix representation of a graph.

15.1.11 What is linear interpolation?

- imputation method which assumes a linear relationship between missing and non-missing values.
- Assume you have three points, A, B , and X with features a and b (all have the same features). for A and B , the value of a and b are known, but for X , only the value of a is known. in this case, the missing value is calculated as the line between A and B , say $f(x)$, such that $f(x) = \frac{A_b - B_b}{A_a - B_a}x + c$ and c is some constant (intercept or bias). Then, X_b is approximated by $f(X_a) \sim X_b$
- Alternatively, some more specific linear interpolation polynomial can be specified.

15.2 Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting

In this paper, S. Guo (2019) proposed a method for traffic flow prediction which utilized an **attention based spatial-temporal graph convolutional network**. They aimed to model several time-dependencies: (1) recent, (2) daily-periodic, and (3) weekly-periodic dependencies. The *attention mechanism* captures spatial-temporal patterns in the traffic data and the *spatial-temporal convolution* is used to capture spatial patterns while *standard convolutions* describe temporal features.

15.2.1 Core Contributions of ASTGCN

Difficulties of the traffic forecasting problem

- it is difficult to handle unstable and nonlinear data
- prediction performance of models require extensive feature engineering
 - domain expertise is necessary
- cnn - spatial feature extraction from grid-based data, gcn - describe spatial correlation of grid based data
 - fails to simultaneously model spatial temporal features and dynamic correlations of traffic data
 - The convolution component of the network is meant to capture the neighborhood feature context surrounding each node - in a spatial temporal model, this means that a three dimensional kernel is required to account for the time dimension
 - the spatial-temporal influence of different edges / connections between nodes varies across different time points.

Addressing these issues:

- develop a *spatial-temporal attention mechanism*
 - learns dynamic spatial-temporal correlations of traffic data
 - temporal attention is applied to capture dynamic temporal correlations for different times
 - the attention mechanism learns the
- Design of *spatial-temporal convolution module*
 - has graph convolution for modeling graph structure
 - has convolution in temporal dimension (kind of like 3-d convolution)
 - the output of the model is a label (for the state of traffic flows) at each node in the traffic network

Related Work:

- two main types of convolutions for graph data
 - spatial methods - directly perform convolution filters on a graph's nodes and their neighbors
 - * Niepert et al. propose a heuristic linear method for selecting the neighborhood of every center node
 - spectral methods -
- *Attention Mechanisms* - mechanisms select information important to current task from all of the inputs.

- Xu et al. -> two mechanisms (image description task) - visualization method used for showing effect of the attention mechanism
- Velickovic et al. -> used “self-attentional layers” for processing graph data by neural networks
- Liang et al. -> multi-level attention network for adaptively adjusting correlations among multiple geographic time series. (it is time consuming to train separate models for each time series though)

15.2.2 Primitives

Traffic networks are defined as undirected graphs $G = (V, E, A)$

- V = set of $|V| = N$ nodes
- E is a set of edges (connectivity between nodes)
- $A \in \mathbb{R}^{N \times N}$ - the adjacency matrix of G
 - F measurements are detected with the same sampling frequency
 - each node has a feature vector of length G at each time slice

15.2.3 Problem Setting

Traffic flow forecasting is formally defined as follows:

- Suppose F time series (states) are recorded for a traffic flow sequence
 - pick the f th time series: $f \in (1, \dots, F)$
 - $x_t^{c,i} \in \mathbb{R}$ - value of c th feature of node i at time t
 - $x_t^i \in \mathbb{R}^F$ - feature vector for node i at time t
 - $X_t = \langle x_t^1, x_t^2, \dots, x_t^N \rangle^T \in \mathbb{R}^{N \times F}$ - set of feature vectors associated with all nodes at time t
 - $\chi = (X_1, X_2, \dots, X_\tau)^T \in \mathbb{R}^{N \times F \times \tau}$ - set of features for all nodes (X_t s) over τ time slices

15.2.4 The ASTGCN Model Framework

There are three independent components to the ASTGCN framework. Each, respectively, is designed for modeling (1) the recent, (2) daily-periodic, and (3) weekly-periodic dependencies of the historical data.

!(The ASTGCN Framework)v2x/imgs/astgcn.png

Each of the three time segments are grouped into segments of size T_h , T_d , and T_w respectively

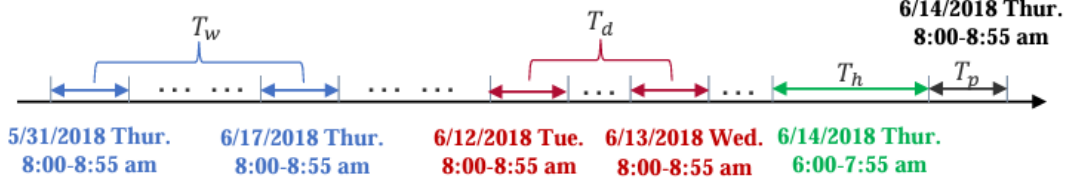


Figure 4: An example of constructing the input of time series segments (suppose the size of predicting window is 1 hour, and T_h , T_d and T_w are twice of T_p).

Figure 15.2: Sample Time Segments of size 1 where $T_h = T_d = T_w = 2$

- Recent time segments: graph states (features for each node in the graph at a given time) $(X_{t_0-T_h+1}, X_{t_0-T_h+2}, \dots, X_{t_0})$. In other words, this segment includes the previous $T_h - 1$ times intervals / series. This includes all of the time series within the segment
- Daily-periodic segments: a series of intervals of length T_p , distributed evenly across T_d/T_p days of length q (q is the number of samples which are measured per day, so to go to the sample which was recorded one day ago at the same time, you need to go to the $t_0 - q + 1$ sample)

$$\begin{aligned}
 & (X_{t_0-(T_d/T_p) \cdot q+1}, \dots, X_{t_0-(T_d/T_p) \cdot q+T_p}, \\
 & X_{t_0-(T_d/T_p-1) \cdot q+1}, \dots, X_{t_0-(T_d/T_p) \cdot q+T_p}, \\
 & X_{t_0-(T_d/T_p-2) \cdot q+1}, \dots, X_{t_0-(T_d/T_p) \cdot q+T_p}, \\
 & \vdots \\
 & X_{t_0-(1) \cdot q+1}, \dots, X_{t_0-(1) \cdot q+T_p} \\
 &)
 \end{aligned}$$

- Weekly-periodic segments:

$$\begin{aligned}
 & (X_{t_0-7 \cdot (T_w/T_p) \cdot q+1}, \dots, X_{t_0-7 \cdot (T_w/T_p) \cdot q+T_p}, \\
 & X_{t_0-7 \cdot (T_w/T_p-1) \cdot q+1}, \dots, X_{t_0-7 \cdot (T_w/T_p) \cdot q+T_p}, \\
 & X_{t_0-7 \cdot (T_w/T_p-2) \cdot q+1}, \dots, X_{t_0-7 \cdot (T_w/T_p) \cdot q+T_p}, \\
 & \vdots \\
 & X_{t_0-7 \cdot (1) \cdot q+1}, \dots, X_{t_0-7 \cdot (1) \cdot q+T_p} \\
 &)
 \end{aligned}$$

NOTE: Although the ASTGCN models and such can be used through PyTorch geometric (library) The original publication and repository includes support for the standard pytorch functions which may be more straightforward to convert into a CrypTen (CrypTensor) model.

15.2.5 Things I don't Understand ...

1. what is the hadammard product?
 1. the hadamard product is simply the element-wise matrix multiplication of two matrices
2. what does it mean to transform the graph convolution setting into the spectral basis, and why is the fourier transform needed to do this? Is it even important for me to answer this question given the setting of the paper itself?
 1. Probably not important. We will need to understand this theory when doing the writeups, for write now, we just need to know the actual polynomial functions used so we can implement them securely.

15.3 Graph WaveNet

Notes for *Graph WaveNet for Deep Spatial-Temporal Graph Modeling* Z. Wu (2019)

15.3.1 Contributions

- Methodology uses a stacked dilated 1D convolution in order to get better temporal representations - longer range time series sequences can be processed effectively
- Automatically learns new graph structures - showing hidden spatial dependencies using a “self-adaptive adjacency matrix”
- [open source git repository](#)

15.3.2 Methods

- Use a graph convolution layer:
 - incorporates graph signal diffusion with K finite steps
 - also utilize an adaptive adjacency parameter with learnable embedding parameter dictionaries for nodes in the graph
- Temporal Convolution Layer:

- uses gating mechanisms and dilated causal convolution networks for handling long sequences
- allows for increased receptive field size (expands range while requiring less depth for the network)
- gated TCN layers are stacked in order to improve handling of spatial dependencies at different temporal levels.

15.3.3 Experiments and Evaluation

- experiments conducted on the METR-LA and PEMS-BAY datasets
- baselines
 - ARIMA - auto-regressive integrated moving average with Kalman filter
 - FC-LSTM - RNN with fully connected LSTM hidden units
 - WaveNet - Conv net for sequence data
 - DCRNN - diffusion conv recurrent neural nets - combine graph convs. with RNNs in encoder-decoder manner
 - GGRU - graph gated recurrent unit network (attention mechanisms used in graph conv.)
 - STGCN - spatial-temporal graph convolution network

15.3.4 Questions

- what is an adaptive dependency matrix (as it relates to the learning of the model?)
 - *receptive field* - region of the input image that is used to compute the output of a particular neuron in the feature map. The receptive field is important because it determines the context or the information that the neuron has access to when making a prediction - see [medium.com](https://medium.com/@davegray/receptive-field-convolutional-neural-networks-400000000000)
- what is the receptive field of a convolution?
- What are “stacked dilated 1D convolutions”? See [WaveNet](#) for more information on this topic
 - *dilated convolutions* - dilated convolutions perform convolutions, but add zeros between the elements of the filter. This expands the receptive field of a given computation by expanding the region of the input covered by a given convolution, but doesn’t increase the number of parameters required to obtain the output. A nice image of this can be seen below:
- What does it mean for the adjacency matrix representation of the graph to be “self-adaptive”, and learnable?
- Why is a node’s representation called a node’s “signal” in this work?
- what is a transition matrix?

Dilated Convolution

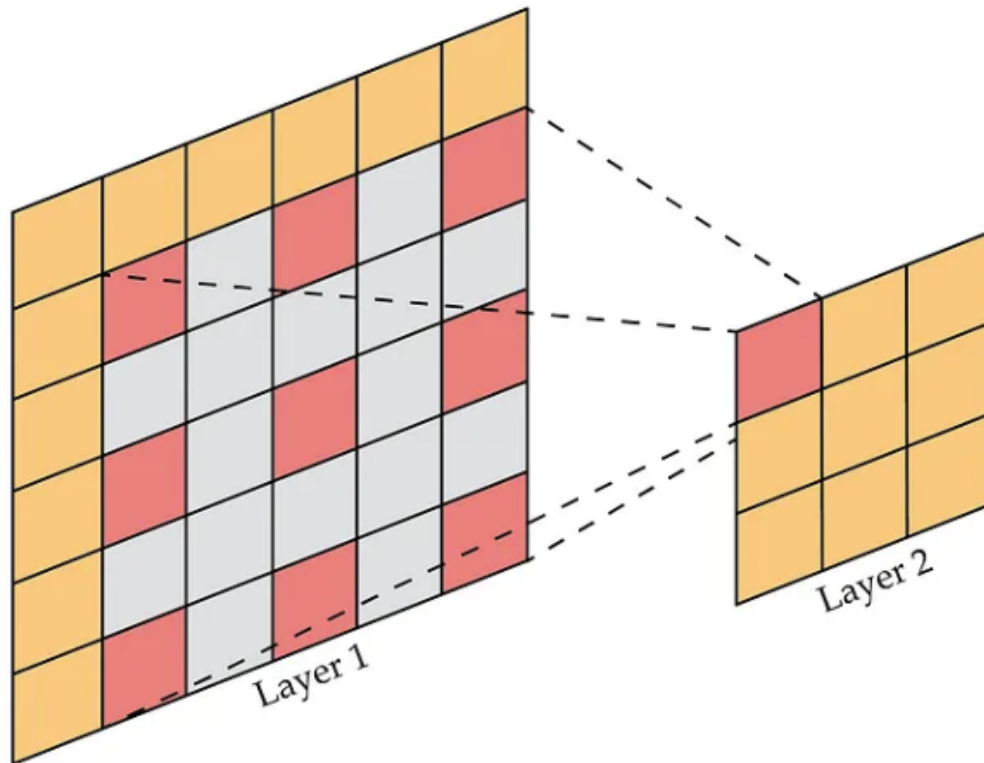


Fig. 2: Dilation convolution — Image created by the author

Figure 15.3: Dilated Convolution Representation

- I believe this idea originates from this paper: [DCNNs](#)
- What is a power series, and how is it applied to a transition matrix?
- What is a TCN layer, let alone a “Gated TCN”? (does this just mean gated temporal convolution?)
- What are gating mechanisms in RNNs, and what are they used for?
- How are graph network structures constructed from the data? See the experimental setup section for more information.
- What is all of this forward-backward adaptive adjacency matrix learning stuff they mention?
 - do they train the graph representation of the traffic flow data separately from the actual graph modeling?

15.3.5 Ideas

- integrate alternative gated TCN functions $g(\cdot)$ in order to compare performance in the secure setting. How can this be used as part of a discussion on theory? Is there even enough time to do this in the project?
- compare the performance of spatio-temporal graph networks along different axes ...
- comparative analysis for graph models - accuracy (MAE/MAPE/RMSE) tradeoffs and running time comparisons (especially for a secure implementation)

15.4 ST-SSL

Use this as a benchmark for everything else. We want to figure out what were the other top models to evaluate. This paper is very recent, and it comes out of a top conference (AAAI)

15.5 STPGCN

[PyTorch repository for this model](#) The Notes from this section are based on the paper: “Spatial-Temporal Position-Aware Graph Convolution Networks for Traffic Flow Forecasting” Y. Zhao (2023)

15.5.1 Contributions

- introduce a spatial-temporal graph representation - models (1) spatial, (2) temporal, and (3) joint spatial-temporal correlations

- develop a position embedding module - encodes spatial-temporal position factors and relation inference module - learn dynamic spatial temporal edge weights
- the authors try to incorporate node-specific patterns and information (from historical data) to improve prediction accuracy at each node. They propose a graph operator, STPGConv for this purpose, as it captures a wide range of spatial, temporal, and joint spatial-temporal correlations
- the network computes spatial-temporal relations in using a pure graph convolution approach. no external temporal components are required, which improves overall efficiency (at least according to their experimental results)
- design an L -layer network based on the STPGConv operation
 - includes skip connections
 - gated selection modules
 - these additions help to overcome problem of vanishing gradient and *feature over-smoothing*
- concatenate features generated by multiple network layers to get an MLP predictor that directly predicts multi-step traffic flows
- typical layer count is $L = 3$. possesses fewer parameters, is computationally efficient

15.5.2 Methods

Zhao et al. propose a novel model and graph convolution architecture for predicting traffic flows based on spatio-temporal information:

Step 1: formulate the graph

- the spatio-temporal graph is represented as a series of β time steps $t, t-1, \dots, t-\beta$ where each time step is represented by a graph $\hat{A} \in \mathbb{R}^{N \times N}$.

$$A_{STP} = \begin{matrix} & t & t-1 & \dots & t-\beta \\ t & \hat{A} & \hat{A} & \dots & \hat{A} \end{matrix}$$

- $\hat{A}$ is an α -hop adjacency matrix of the road network: a picture of these hops can be seen below:
- each element of $\hat{A}$, the adjacency matrix, is given as follows:

$$\hat{A}_{ij} = \begin{cases} 1 & \text{if } \text{SDist}(v_i, v_j) \leq \alpha, \\ 0 & \text{if the shortest path between } v_i \text{ and } v_j > \alpha \end{cases}$$

in other words, if the shortest path between any two nodes in the spatial direction is of a distance less than α , then there is an α -hop edge which connects the two nodes - hence a 1 fills that connection label in the matrix $\hat{A}$ Graph Edge Weights

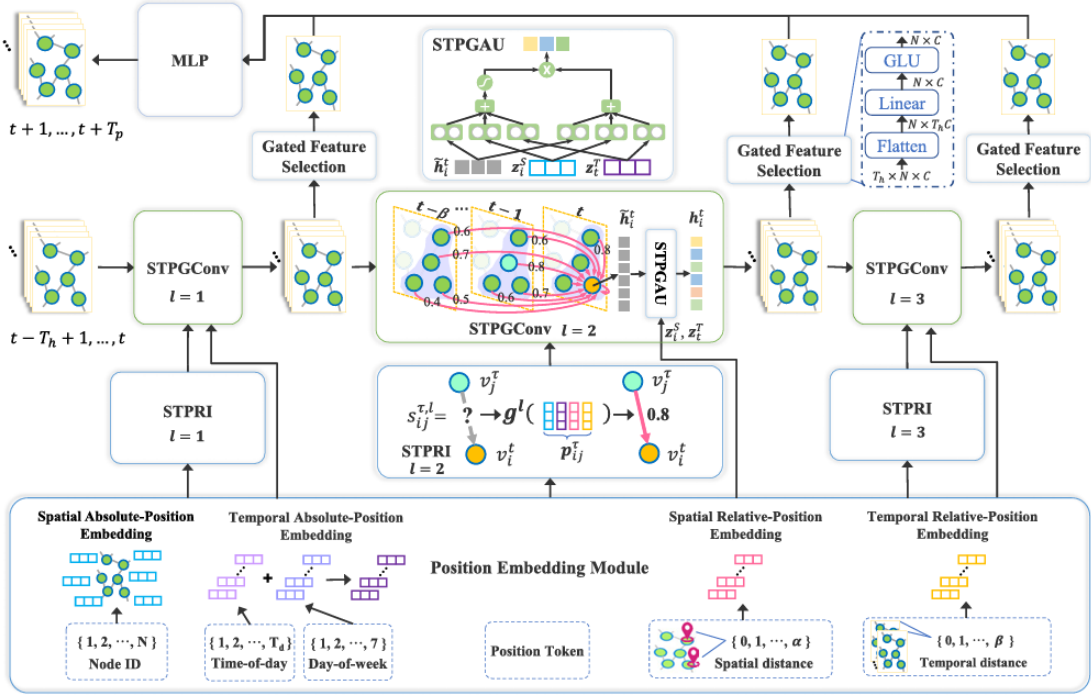


Fig. 5. Illustration of the proposed STPGCN. It works according to the following workflow. At first, the position embedding module generates embedding representations for spatial / temporal absolute-position, and spatial / temporal relative-position. And then, the spatial-temporal position-aware relation inference (STPRI) module infers the spatial-temporal relations between different nodes based on their position embedding. For example, given nodes v_i^t and v_j^t , the STPRI module collects the position embedding p_{ij}^t associated with the edge (v_i^t, v_j^t) and then learns a function $g^l(\cdot)$ to infer the spatial-temporal relation weight $s_{ij}^{t,l}$. Based on this, the learned spatial-temporal relations are used to construct a spatial-temporal position-aware graph convolution layer (STPGConv). In each STPGConv, neighbor node features on continuous time slices $\{t - \beta, \dots, t\}$ are aggregated to the target node v_i^t according to the learned edge weights. In addition, the gated feature selection module adaptively selects important features from the output of each STPGConv layer and sends them directly to the prediction module by skipping connections. Finally, the prediction module uses the MLP to generate future traffic flows directly.

Figure 15.4: The architecture of STPGCN

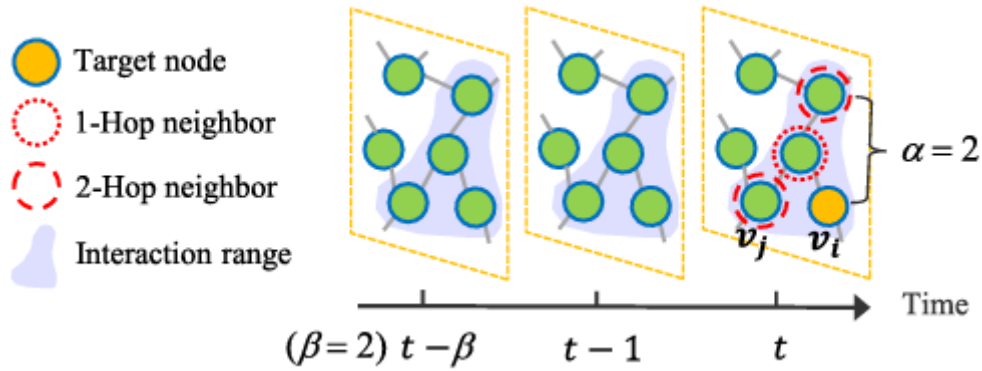


Figure 15.5: spatio-temporal connection range

Step 2: Set edge weights for the unweighted graph

- *spatial and temporal absolute-position embeddings*

- spatial embeddings are defined - represent the absolute position of a given vector

$$z^S = [z_1^S, z_2^S, \dots, z_N^S] \in \mathbb{R}^{N \times d}$$

- temporal embeddings are defined similarly

$$z^T = [z_1^T, z_2^T, \dots, z_N^T] \in \mathbb{R}^{T_d \times d}$$

where $Z_t^T = z_m^D + z_n^W$.

- * z_m^D - the temporal position embedding for the day - $\{1, 2, \dots, T_d\}$
- * z_n^W - the temporal position embedding for the week- $\{1, 2, \dots, 7\}$
- * these are summed together to obtain the temporal position embedding

- *spatial and temporal relative-position embeddings*

- these embeddings encode the relative distances between two nodes
- relative spatial and temporal distances between two nodes given as the shortest paths between them across each dimension:

$$\text{SDist}(\cdot, \cdot) \rightarrow a \in \{0, 1, \dots, \alpha\}$$

$$\text{TDist}(\cdot, \cdot) \rightarrow b \in \{0, 1, \dots, \beta\}$$

- the temporal distance can be no more than β , and the spatial distance can be no greater than α .
- when $a = 0$, the two nodes are in the same position (spatially)
- when $b = 0$, the two nodes are in the same time slice
- distance values, a and b -> discrete label tokens and generate trainable embedding vectors
- Spatial and temporal relative-position embedding vectors generated as:

$$Z^{SD} = [z_0^{SD}, z_1^{SD}, z_2^{SD}, \dots, z_\alpha^{SD}] \in \mathbb{R}^{(\alpha+1) \times d}$$

$$Z^{TD} = [z_0^{TD}, z_1^{TD}, z_2^{TD}, \dots, z_\beta^{TD}] \in \mathbb{R}^{(\beta+1) \times d}$$

- it's not obvious to me what the d symbolizes ...

- *spatial-temporal position-aware relation inference*

- this module takes the embeddings generated for each node, and uses them to map the value of weights for each edge in the graph (dynamic - can change for different time steps)
- you have a node v_i^t with candidate neighbors v_j^τ in the interaction range $i, j \in \{1, 2, \dots, N\}, t \in \{1, 2, \dots, T_h\}, \tau \in \{t - \beta, t - \beta + 1, \dots, t\}$ and $t - \beta > 0$
- $S^t = \{s_{ij}^\tau | i, j = 1, \dots, N; \tau = t - \beta, \dots, t\} \in \mathbb{R}^{N \times (\beta+1) \times N}$

- * denote dynamic weights for all spatio-temporal edges
- * $A_{STP}^t = S^t \odot A_{STP}$ - element-wise multiplication of the edge weights by the adjacency matrix.
- * each element of S^t , s_{ij}^τ is given by

$$s_{ij}^\tau = \begin{cases} g(p_{ij}^\tau) & \text{if } \text{SDist}(v_i^t, v_j^\tau) \leq \alpha \\ 0 & \text{otherwise} \end{cases}$$

- * p_{ij}^τ is the position feature vector
- * $p_{ij}^\tau = [z_i^S || z_j^S || z_i^T || z_j^T || z_a^{SD} || z_b^{SD}] \in \mathbb{R}^{6d}$
 - z_i^S, z_j^T - spatial / temporal position info of node v_i^t
 - z_j^S, z_i^T - spatial / temporal position information of node v_j^τ
 - z_a^{SD}, z_b^{SD} - spatial / temporal distance between v_i^t and v_j^τ
- * $g(\cdot)$ is a learnable Gaussian kernel for mapping weights from embedding vector inputs
- * $g(p_{ij}^\tau) = \exp\left(-\frac{1}{2}(p_{ij}^\tau - \mu)^T \Sigma^{-1}(p_{ij}^\tau - \mu)\right)$
 - $\Sigma = \text{diag}(\sigma)$ - trainable diagonal covariance matrix
 - $\mu \in \mathbb{R}^{6d}$ - trainable mean vector
 - $\sigma \in \mathbb{R}^{6d}$ - trainable variance vector

Step 3: Perform Spatial-Temporal Position-Aware Graph Convolution

- *joint spatial-temporal node feature aggregator STPGConv*
 - in each STPGConv layer, the dynamic spatial-temporal edge weights are generated based on the position embedding p_{ij}^τ
 - weight summation function is constructed:

$$\tilde{H}^{t,l} = \phi_{stpgcn}(A_{STP}^{t,l}, \hat{H}^{t,l-1}) = S^{t,l} \hat{H}^{t,l-1}$$

where $\hat{H}^{t,l-1} = [H^{t,l-1}; \dots; H^{t-\beta,l-1}] \in \mathbb{R}^{(\beta+1)N \times C}$

- * represent node features output from layer $l-1$ on $\beta+1$ time slices
- The discrete STPGCN operation is given by

$$\begin{aligned} \tilde{h}_i^{t,l} &= \phi_{stpgcn}(s_{ij}^{\tau,l}, h_j^{\tau,l-1}) \\ &= \sum_{j=1}^N \sum_{\tau=t-\beta}^t s_{ij}^{\tau,l} \cdot h_j^{\tau,l-1} \end{aligned}$$

- * $s_{ij}^{\tau,l}$ - weight of edge connecting nodes v_i^t and v_j^τ (in the l th layer)
- * $h_j^{\tau,l-1}$ - the features of node v_j^τ from the previous layer

- A linear layer is applied on $\tilde{h}_i^{t,l}$ to obtain the traffic flow signal of node $v_j^\tau, h_j^{\tau,0}$

$$h_j^{\tau,0} = W^0 \cdot x_j^\tau + b^0$$

$$h_i^{t,l} = \Psi(W^l \cdot \tilde{h}_i^{t,l} + b^l)$$

* Ψ - the activation function

- *spatial-temporal position-aware gated activation unit STPGAU*

- gating activation units are used to maintain the feature patterns across different time steps for individual nodes.
- formulation:

$$\begin{aligned} h_i^{t,l} &= \text{STPGAU}(\tilde{h}_i^{t,l}, z_i^S, z_t^T) \\ &= (W_1^l \cdot \tilde{h}_i^{t,l} + W_2^l \cdot z_i^S + W_3^l \cdot z_t^T + b_1^l) \\ &\odot \delta(W_4^l \cdot \tilde{h}_i^{t,l} + W_5^l \cdot z_i^S + W_6^l \cdot z_t^T + b_2^l) \end{aligned}$$

* the spatial and temporal embedding vectors are incorporated into the sums in order to maintain the individual patterns and post-tune the output of the STPGConv layer to generate node-specific pattern features

Step 4: Gated Feature Selection and Skip Connections

- avoid gradient vanishing problem by sending outputs of each STPGConv layer to the output via *skip connections*
- a gated feature selection module is used to select (adaptively) important features from multiple layers for the final prediction to avoid the *over-smoothing problem*

Step 5: Make Predictions

- an MLP is used to generate future multi-step results directly - this is done using inputs from the L layers of features generated by the network

$$U = \hat{H}^1 || \dots || \hat{H}^L$$

- these inputs are fed to the MLP to make predictions

15.5.3 Evaluation

15.5.4 Questions

- I still don't know what "Gated Activation Units" are

16 Trajectory-Based Traffic Forecasting

16.1 T-Wave

Notes for T-Wave introduced in B. Hui (2021)

16.1.1 Contributions

16.1.2 Methods

16.1.3 Experiments

16.1.4 Questions

- How are trajectory sequences / data actually constructed as inputs to the model?
- How can I obtain access to the code for these models?
- How are the traffic segments structured? They utilize the same methods of dilate spatial temporal convolutions as Graph-WaveNet, but use the trajectory data in order to structure the actual graph data. I don't really understand their representations too well.

16.2 MA2GCN

Notes on “MA2GCN: Multi Adjacency Relationship Attention Graph Convolutional Networks for Traffic Prediction Using Trajectory Data” Z. Sun (2024)

16.2.1 Contributions

- authors provide methods for converting trajectory-based datasets for traffic flows prediction into graph networks.
- they also provide code implementations
- their graph constructions consider entry and exit matrices for the different grids they construct.

- Their graph analysis framework is extremely similar to graph wavenet, but their adaptive graph generator is unique:
- multi-adjacency relationship attention?

16.2.2 Methods

- utilizes a multi-adjacency relationship attention module to try and accurately learn the spatial information of the input
 - focuses on the more important adjacency matrices which are input?
 - how many adjacency matrices are input to the model? shouldn't there be a single matrix representing the entire graph / range of the data or do they divide the data / area of coverage into separate grids and then input them together? The process used here is not very clear.

16.2.3 Experiments / Evaluation

- They use the Shanghai taxi GPS trajectory dataset, and convert it into a graph using their novel trajectory-to-graph conversion methods
- they test their graph convolution routines against state-of-the-art methods for these converted graph data.
- they use the same metrics that all of the GCN analysis papers use. I don't really like their writeups though. I'm souring on this work a bit. It would be better if we could get access to T-Wave, then we could perform some better analysis on a better model potentially.

16.2.4 Questions

- How is the actual conversion performed? I.e., how does the trajectory data look to start, and how is it converted to a graph format.
- What is the $\tilde{X}$ matrix they mention in the adaptive graph generator section?
- what is the notation?
 - P, D, N, C - what do these symbols denote?

17 Summary

In summary, this book has no content whatsoever.

References

- B. Hui, H. Chen, D. Yan. 2021. “Trajectory WaveNet: A Trajectory-Based Model for Traffic Forecasting.” *IEEE International Conference on Data Mining*. <https://ieeexplore.ieee.org/document/9679147>.
- B. Knott, A. Hannun, S. Venkataraman. 2022. “CrypTen: Secure Multi-Party Computation Meets Machine Learning.” <https://arxiv.org/abs/2109.00984>.
- B. Yu, Z. Zhu, H. Yin. 2018. “Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting.” *IJCAI*. <https://doi.org/10.24963/ijcai.2018/505>.
- Beaver, D. 1991. “Efficient Multiparty Protocols Using Circuit Randomization.” *Advances in Cryptology*. https://link.springer.com/chapter/10.1007/3-540-46766-1_34.
- . 1996. “Correlated Pseudorandomness and the Complexity of Private Computations.” *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, 479–88. <https://dl.acm.org/doi/pdf/10.1145/237814.237996>.
- Dokur, O., and S. Katkouri. 2022. “Vehicle-to-Infrastructure Based Algorithms for Traffic Light Detection, Red Light Violation, and Wrong-Way Entry Applications.” *IEEE International Symposium on Smart Electronic Systems*.
- E. Boyle, N. Gilboa, G. Couteau. 2020. “Correlated Psuedorandom Functions from Variable-Density Lpn.” *Cryptology ePrint Archive*. <https://eprint.iacr.org/2020/1417.pdf>.
- J. Goma, R. Bautista, M. Eviota. 2020. “Detecting Red-Light Runners (RLR) and Speeding Violation Through Video Capture.” *IEEE 7th International Conference on Industrial Engineering and Applications*.
- J. Terven, D. Cordova-Esparaza. 2023. “A Comprehensive Review of YOLO: From YOLOv1 to YOLOv8 and Beyond.” *ACM Computing Surveys*. <https://arxiv.org/pdf/2304.00501v1.pdf>.
- K. Yang, X. Lan, C. Weng. 2020. “Ferret: Fast Extension for Correlated Ot with Small Communication.” *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 1607–26. <https://eprint.iacr.org/2020/924.pdf>.
- Killian, J. 1988. “Founding Cryptography on Oblivious Transfer.” <https://dl.acm.org/doi/pdf/10.1145/62212.62215>.
- L. Thao, N. Anh, D. Cuong. 2022. “Automatic Traffic Red-Light Violation Detection Using AI.” *International Information and Engineering Technology Association*.
- Maindola, G. 2021. “Introduction to YOLOv5 Object Detection with Tutorial.” <https://machinelearningknowledge.ai/introduction-to-yolov5-object-detection-with-tutorial/>.
- O. Golrich, R. Vanish. 1987. “How to Solve Any Protocol Problem - an Efficiency Improvement.” https://link.springer.com/content/pdf/10.1007/3-540-48184-2_6.pdf.

- R. Impagliazzo, S. Rudich. 1989. “Limits on the Provable Consequences of One-Way Permutations.” *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing*, 44–61. <https://dl.acm.org/doi/pdf/10.1145/73007.73012>.
- S. Guo, N. Feng, Y. Lin. 2019. “Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting.” *The Thirty-Third AAAI Conference on Artificial Intelligence*. <https://dl.acm.org/doi/pdf/10.1609/aaai.v33i01.3301922>.
- W. Liu, D. Erhan, D. Anguelov. 2016. “SSD: Single Shot MultiBox Detector.” *ECCV*.
- Wong, W. 2021. “What Is Residual Connection?” <https://towardsdatascience.com/what-is-residual-connection-efb07cab0d55>.
- Y. Ishai, K. Nissin, J. Kilian. 2003. “Extending Oblivious Transfers Efficiently.” *Annual International Cryptology Conference*, 145–61. https://link.springer.com/content/pdf/10.1007/978-3-540-45146-4_9.pdf.
- Y. Zhao, H. Wen, Y. Lin. 2023. “Spatial-Temporal Position-Aware Graph Convolution Networks for Traffic Flow Forecasting.” *IEEE Transactions on Intelligent Transportation Systems*. <https://ieeexplore.ieee.org/abstract/document/9945663>.
- Z. Sun, Y. Ma. 2024. “MA2GCN: Multi Adjacency Relationship Attention Graph Convolutional Networks for Traffic Prediction Using Trajectory Data.” *arXiv Preprint*. <https://arxiv.org/pdf/2401.08727>.
- Z. Wu, G. Long, S. Pan. 2019. “Graph WaveNet for Deep Spatial-Temporal Graph Modeling.” *IJCAI*. <https://doi.org/10.48550/arXiv.1906.00121>.